Movientum: A Full-Stack Personalized Movie and TV Recommendation Platform with a Multi-Tier Graph-Augmented Learning-to-Rank Engine

System Design, Machine Learning Architecture, and Implementation Documentation

Movientum Engineering Documentation
System Architecture, Backend, Frontend, and Machine Learning Design Report
Prepared as a Complete Technical Reference for the Movientum Platform

***Abstract*—Movientum is a full-stack, cinematic movie and television discovery platform that combines a modern React single-page application with a FastAPI-based modular monolith backend and a six-tier, graph-augmented, learning-to-rank recommendation engine. Unlike conventional star-rating or ten-point movie platforms, Movientum introduces a proprietary four-category emotional-intent rating system (*Skip*, *Timepass*, *Go For It*, *Perfection*) that feeds directly into a real-time user taste-profile vector store. The recommendation subsystem fuses a bipartite content graph traversed via Personalized PageRank (Random Walk with Restart) with an XGBoost `XGBRanker` learning-to-rank model trained nightly on interaction logs, and blends the resulting candidates with baseline popularity models using Team-Draft Interleaving. The platform is backed by Supabase PostgreSQL for durable storage, Upstash Redis for sub-millisecond caching and stampede protection, Celery for asynchronous task orchestration, and Azure Monitor OpenTelemetry for observability. This document provides a complete, self-contained technical specification of the system: the system architecture, database schema, authentication design, search infrastructure, ratings and feedback logic, watchlist and collections model, news aggregation pipeline, caching strategy, containerization and deployment topology, scalability plan, storage capacity estimation, and the full mathematical and algorithmic treatment of the recommendation engine, including graph construction, edge-weight rationale, feature engineering, ranking objective, ensemble blending, and the nightly retraining pipeline. Workflow diagrams, entity-relationship tables, sequence diagrams, and algorithmic pseudocode are provided throughout so that any reader, irrespective of prior exposure to the codebase, may understand both the engineering rationale and the precise operational behavior of every subsystem.**

***Index Terms*—recommendation systems, learning-to-rank, XGBoost, graph neural retrieval, personalized PageRank, random walk with restart, FastAPI, PostgreSQL, Redis caching, JWT authentication, full-text search, modular monolith, Celery, microservices alternative, content-based filtering, collaborative signals, team-draft interleaving**

I. Introduction

A. Motivation

Modern streaming and cataloging platforms overwhelmingly rely on either a five-star or ten-point numerical rating scale to elicit user preference signals. Such scales are ambiguous: a "7/10" from one user may represent enthusiastic endorsement, while the same numeric value from another user may represent lukewarm indifference. Movientum was designed from first principles to reject this ambiguity in favor of a small, emotionally unambiguous, categorical rating vocabulary, while simultaneously building a state-of-the-art, low-latency, personalized recommendation pipeline on top of that signal.

B. Design Goals

The system was engineered against five concrete goals:

G1. **Unambiguous Preference Capture** — replace numeric ratings with a four-category system that captures both quality judgment and rewatchability/context intent.

G2. **Real-Time Personalization** — every explicit or implicit signal (thumbs up/down, poster click, scroll-ignore, watch, watchlist add) must update the user's taste profile immediately, with recommendations reflecting the change on the very next request, not after a batch job.

G3. **Operational Simplicity** — avoid the overhead of a microservice mesh; instead build a modular monolith that keeps the in-memory recommendation graph co-located with the API process to avoid network round-trips during traversal.

G4. **Cost-Efficient Infrastructure** — rely on managed, serverless-friendly infrastructure (Supabase PostgreSQL, Upstash Redis) rather than self-hosted database or cache clusters, and avoid heavyweight search infrastructure (Elasticsearch/Meilisearch) in favor of native PostgreSQL full-text search.

G5. **Explainable, Auditable Machine Learning** — use a graph-based candidate generator (NetworkX bipartite graph + Personalized PageRank) combined with a gradient-boosted learning-to-rank model (XGBoost `XGBRanker`) rather than an opaque deep neural recommender, so that every recommendation can be traced back to specific graph edges and feature contributions.

C. Document Structure

Section II presents the overall system architecture and the four architectural pillars. Section III enumerates the complete technology stack. Section IV describes the backend modular monolith design. Section V describes the React single-page-application frontend. Section VI details the JWT authentication system. Section VII covers the PostgreSQL database architecture and schema. Section VIII explains the

configuration management design. Section IX documents the four-category rating system. Section X documents the PostgreSQL-native search and filtering subsystem. Section XI documents the multi-collection watchlist feature. Section XII through Section XV provide the complete recommendation engine documentation across all six tiers, including graph construction, feature engineering, XGBRanker inference, content-based basket recommendations, and the nightly retraining pipeline. Section XVI documents the news aggregation and personalization pipeline. Section XVII documents the Redis caching layer in full detail. Section XVIII documents containerization. Section XIX documents the multi-server deployment topology. Section XX documents the scalability plan. Section XXI documents storage capacity estimation. Section XXII documents the observability and telemetry stack. Section XXIII provides the complete API endpoint reference. Section XXIV provides the annotated codebase map. Section XXVIII concludes.

II. Master System Architecture

A. Architectural Philosophy

Movientum operates as a **Modular Monolith**: a single, well-factored backend codebase with strict internal boundary separations (routers, services, repositories, ML modules) rather than a distributed microservice mesh. This decision was driven primarily by the recommendation engine's need for an in-memory `networkx.Graph` singleton. Distributing this graph across microservices would require either (a) a distributed graph database such as RedisGraph, introducing network latency into every Random-Walk-with-Restart traversal, or (b) serializing/deserializing the graph across service boundaries on every request — both unattractive relative to simply keeping the graph resident in the API server's own process memory and scaling the API horizontally by duplicating that graph into each new worker's RAM.

B. The Four Pillars

The system is organized into four cooperating tiers:

1) **The Client Tier** — a React 19, Client-Side-Rendered (CSR) Single Page Application, statically hosted on a global CDN (Vercel Edge Network).
2) **The API Tier** — a stateless FastAPI backend container, horizontally scaled behind a load balancer, responsible for authentication, database access orchestration, and real-time ML inference.
3) **The Worker Tier** — Celery worker and beat-scheduler containers responsible for TMDB data ingestion and nightly machine-learning retraining.
4) **The Data Tier** — Supabase (managed PostgreSQL) for durable persistence and Upstash (managed, serverless Redis) for caching, JWT blacklisting, and the Celery message broker.

C. Full System Diagram

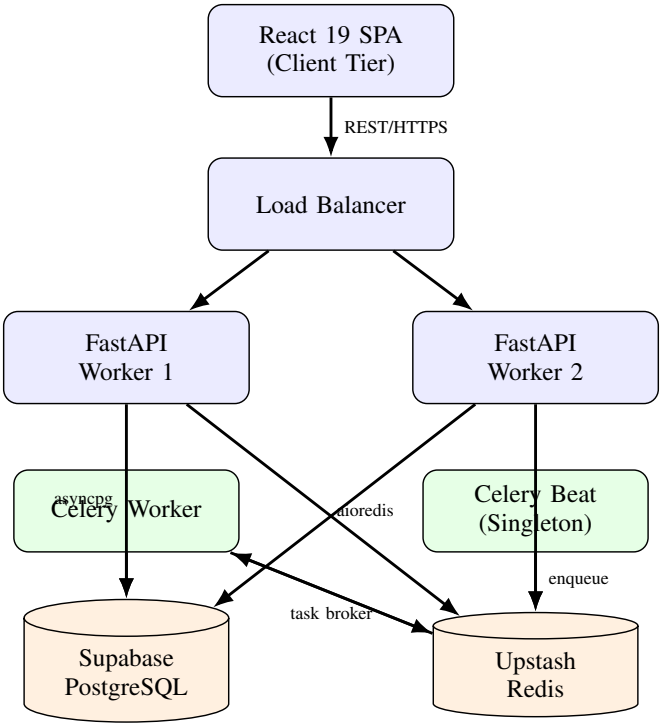

Fig. 1: Full system architecture: client, API, worker, and data tiers.

D. Architecture Component Responsibility Table

E. Component Interaction Summary

The Frontend communicates with the backend exclusively via a REST API built on `axios`; it maintains no direct database connection of any kind, which keeps all business logic, validation, and security enforcement centralized in the backend. The FastAPI backend acts as the sole orchestrator for the recommendation pipeline: for every recommendation request it (1) queries the database for the user's taste profile, (2) queries the RAM-resident NetworkX graph for graph-based candidates, (3) executes XGBRanker inference over the resulting feature matrix, and (4) interleaves the ranked output with a baseline model before returning JSON to the client. Celery's beat scheduler independently triggers a nightly job that pulls the last 30 days of interaction logs from the database, retrains the XGBoost ranking model, and persists the resulting `ranker.json` artifact to local disk, which the live FastAPI workers then hot-reload without requiring a restart.

TABLE I: Architecture Component Map

Component	Responsibility	Scaling Model
Vercel Edge	Serve static HTML/JS/CSS	Automatic Edge Scaling
FastAPI Containers	Handle HTTP requests, run ML inference	Horizontal (Load Balanced)
Celery Worker	TMDB Ingestion, Background tasks	Horizontal (Queue Depth)
Celery Beat	Trigger nightly cron jobs	Singleton (Strictly 1 Instance)
Supabase DB	Store users, catalogs, profiles, logs	Vertical (Compute/Storage)
Upstash Redis	Cache API responses, Celery Broker	Serverless (Usage Based)

III. Technology Stack

A. Frontend Technologies

TABLE II: Frontend Technology Stack

Technology	Role
React 19 + Vite 8	SPA framework and build tool
React Router DOM 7	Client-side routing
Axios	HTTP client with JWT interceptors
OGL	WebGL aurora background animation
Motion (Framer Motion)	Layout transitions, micro-interactions
Vanilla CSS	Custom design system, glassmorphism, animation

Deployment target: **Vercel** (global CDN, edge network).

B. Backend Technologies

TABLE III: Backend Technology Stack

Technology	Role
FastAPI + Python 3.13	Asynchronous REST API framework
SQLAlchemy 2 + Alembic	Async ORM and schema migration tooling
asyncpg	Async PostgreSQL driver
Celery	Background task queue
python-jose + passlib[bcrypt]	JWT authentication and password hashing
Pydantic v2	Request and response schema validation

Deployment target: **Azure** with Docker containers.

C. Infrastructure and Data Technologies

TABLE IV: Infrastructure and Data Technology Stack

Service	Role
PostgreSQL 15+ (Supabase)	Primary relational data store
Redis 7+ (Upstash)	Cache, JWT blacklist, Celery broker
Nginx	Reverse proxy, SSL termination, rate limiting
TMDB API	Movie/TV metadata, imagery, trending data
NewsAPI	Movie-related news article ingestion

D. Machine Learning Technologies

TABLE V: ML and Recommendation Technology Stack

Technology	Role
NetworkX + scipy.sparse	Bipartite content graph and RWR traversal
XGBoost (XGBRanker)	Learning-to-rank model, NDCG objective
NumPy	Feature matrix construction

IV. Backend Architecture: Modular Monolith

A. Design Rationale

Movientum's backend is a strictly bounded Modular Monolith built on FastAPI. It deliberately avoids microservice decomposition overhead while preserving clean internal boundaries within a single codebase and a single deployable Docker image. Three concrete rationales justify this decision:

- **Shared Data Models**: The ML ingestion loop and FastAPI routers share SQLAlchemy ORM models directly, with no gRPC or serialization boundary between them.
- **Simpler Deployments**: A single Docker image (or a horizontally-scaled cluster of identical containers) scales the whole API tier behind one load balancer.
- **Graph Cache Locality**: The `networkx.Graph` recommendation singleton must live in process RAM; a microservice split would force either a distributed graph store or repeated network calls per traversal step, both far slower than in-process access.

B. Directory Organization

The backend source tree under `backend/app/` is organized as follows:

- `routers/` — FastAPI endpoint definitions, one file per feature domain (e.g. `movies.py`, `recommendations.py`, `users.py`).
- `services/` — All core business logic, isolating routers from raw database queries (e.g. `tmdb_service.py`, `feedback_service.py`, `advanced_recs.py`).
- `db/` — Connection pooling (`database.py`), ORM schema definitions (`orm_models.py`), and Redis connection/key management (`cache.py`).
- `ml/` — XGBRanker inference wrapper (`ranker.py`) and the Celery-triggered training loop (`training.py`).
- `main.py` — Application entry point; initializes routers, CORS policy, and the OpenTelemetry exporter.

C. Dependency Injection Pattern

FastAPI's `Depends` mechanism is used pervasively to inject async database sessions and the authenticated user's identity into route handlers, keeping every component independently testable and free of global mutable state.

```
@router.post("/feedback/")
async def submit_feedback(
    payload: FeedbackRequest,
    db: AsyncSession = Depends(get_db),
    user_id: str = Depends(get_current_user_id)
```

```
):
    await apply_feedback(db, user_id, payload.tmdb_id,
     payload.signal_type)
```

Listing 1: Dependency injection example for feedback submission

D. Core Backend Stack Summary

TABLE VI: Core Backend Technology Responsibilities

Technology	Role
FastAPI	Asynchronous web framework
SQLAlchemy 2.0	Asynchronous ORM (`asyncpg`)
Pydantic V2	Request validation and settings management
Celery	Distributed task queue for async jobs
NetworkX	In-memory bipartite graph processing
XGBoost	Learning-to-Rank ML inference (`XGBRanker`)

E. Request Lifecycle

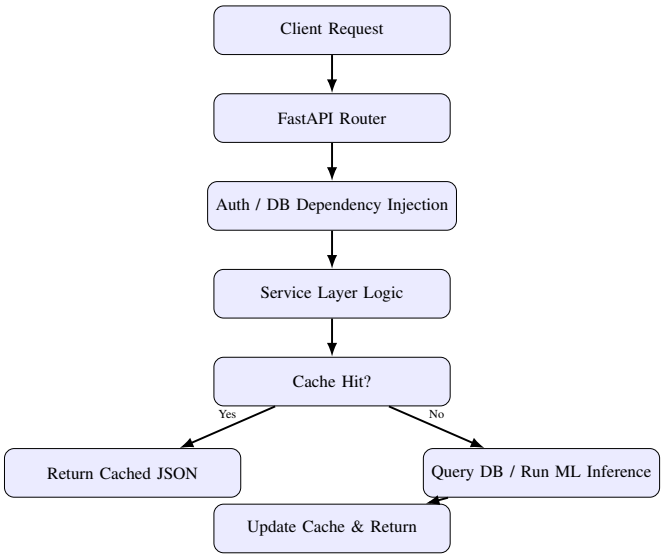

Fig. 2: Backend request lifecycle with cache-first strategy.

V. Frontend Architecture: React Single Page Application

A. Rendering Strategy

The Movientum frontend is a purely Client-Side-Rendered (CSR) Single Page Application, deliberately avoiding meta-frameworks such as Next.js. The rationale is that more than 90% of content views inside Movientum are unique to the authenticated user's individualized taste profile, meaning that Server-Side Rendering (SSR) would provide minimal caching benefit while adding rendering-server complexity. Pure CSR allows the interface to react instantly to interactions — such as a thumbs-up action — using local React state, while the corresponding write is dispatched asynchronously to the backend.

B. Aesthetic and Animation Requirements

The interface targets a premium, cinematic visual identity through two specialized libraries:

- **Motion (Framer Motion)**: layout transitions, micro-interactions, and presence/exit animation detection.
- **OGL**: a minimal WebGL library driving the animated Aurora background effect on the introductory landing page.

C. Key Frontend Dependencies

```
"dependencies": {
  "react": "^19.2.6",
  "react-dom": "^19.2.6",
  "react-router-dom": "^7.15.1",
  "motion": "^12.40.0",
  "ogl": "^1.0.11",
  "axios": "^1.16.1",
  "react-icons": "^5.6.0"
}
```

Listing 2: Frontend package.json dependencies (excerpt)

D. Build Tooling

Vite is used for both lightning-fast Hot Module Replacement (HMR) during development and highly optimized Rollup-based production bundling. ESLint is strictly enforced with the React Hooks and React Refresh plugin rule sets.

E. Key Technology Table

TABLE VII: Frontend Key Technologies

Technology	Purpose
React 19	Core UI library
Vite	Build tool and development server
React Router v7	Client-side routing
Axios	Promise-based HTTP client to FastAPI backend
Motion	Fluid animation and gesture support

F. Frontend Data Flow

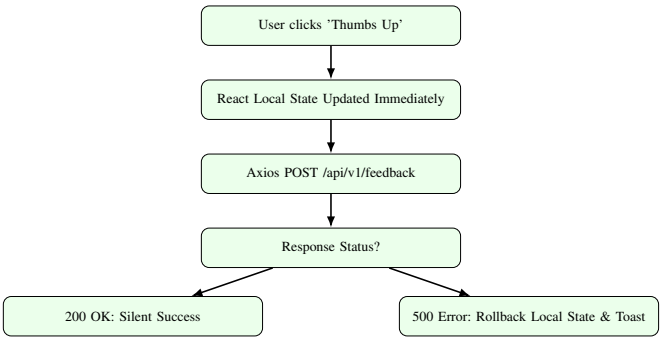

Fig. 3: Optimistic UI update flow for real-time feedback signals.

G. Frontend Directory Layout

The frontend is organized under `frontend/src/` into: `context/` (global auth provider), `services/` (Axios request wrappers, one per domain), `pages/` (route-level page components with matching CSS), `hooks/` (custom

React hooks such as scroll restoration and session-backed state), `utils/` (Axios instance with interceptors, in-memory page cache, LocalStorage helpers, analytics queue), and `components/` (reusable presentational widgets such as the movie card, rating meter, search overlay, and navbar).

VI. Authentication System

A. Overview

Movientum utilizes a stateless, JWT-based (JSON Web Token) authentication system. This design allows the API to scale horizontally without any server-side session store, since every request carries its own cryptographically verifiable identity claim. Passwords are hashed with `bcrypt` prior to persistence in the Supabase PostgreSQL database; raw passwords are never written to disk or logs.

B. Token Lifecycle

Two token types govern the authentication lifecycle:

1) **Access Token** — short-lived (default 48 hours). Carried in the `Authorization: Bearer <token>` header on every protected request.
2) **Refresh Token** — long-lived (default 7 days). Stored client-side and exchanged at `/api/v1/auth/refresh` to mint a new Access Token without forcing re-authentication.

C. Password Security

The `passlib[bcrypt]` library generates a securely salted hash at registration time. At login time, the submitted plaintext password is hashed using the same algorithm and compared, in constant time, against the stored hash. At no point does a raw password touch persistent storage.

D. Dependency-Injected Authorization

FastAPI's dependency injection system secures protected endpoints via the `get_current_user` dependency, which automatically: extracts the bearer token from the request header, decodes the JWT payload, verifies its signature against `jwt_secret_key`, and checks token expiration — raising `401 Unauthorized` on any failure.

E. Core Components

- `app/utils/security.py` — `verify_password`, `get_password_hash`, `create_access_token`, `create_refresh_token` (built on `PyJWT`).
- `app/utils/deps.py` — the `get_current_user` FastAPI dependency.
- `app/routers/auth.py` — the REST surface described below.

F. Authentication Endpoint Reference

Note that logout in the base design is client-side (token deletion from local storage), with the server *optionally* blacklisting the token via Redis; the README additionally documents an immediate Redis-based logout blacklist as a security hardening on top of this base flow, ensuring that a token cannot be reused even within its remaining validity window after explicit logout.

G. JWT Request Flow

VII. Database Architecture (Supabase PostgreSQL)

A. Overview

Movientum uses Supabase PostgreSQL as its sole primary operational data store, accessed through SQLAlchemy 2.0 in fully asynchronous mode (`asyncpg`) at runtime, while Alembic migrations use the synchronous `psycopg2` driver.

B. Schema Hierarchy

The schema, defined in `backend/app/db/orm_models.py`, is organized into three logical phases:

1) **Phase 1 — Master Catalog**: `movies`, `genres`, `directors`, and `ContentCatalog` — a fast-inference structure using `ARRAY(Integer)` and `JSONB` columns to hold TMDB categorical and talent features for ML consumption.
2) **Phase 3 — Auth and Identity**: `users` (UUID primary keys, chosen specifically to prevent sequential-ID enumeration attacks), and `UserTasteProfile` (six multi-dimensional JSONB weight vectors: genres, cast, crew, keyword, language, era).
3) **Phase 6 — Logging and Feedback**: `ratings` (four fixed categories), and `InteractionLog` (stores the exact 16-dimensional ML feature snapshot present at the moment a user clicked or thumbed an item, which becomes the nightly training dataset).

C. Key Tables Overview

TABLE IX: Key Database Tables

Table	Purpose
users	Accounts (UUID PK, bcrypt hashed password)
movies	TMDB movie catalog with FTS vector
ratings	4-category ratings (upsert per user/movie)
watch_history	Watch records
watchlist	Saved-for-later items
content_catalog	20K+ items with genre/keyword/cast/crew/era arrays
user_taste_profiles	Per-user JSONB affinity weight vectors
interaction_log	Click/thumbs events used for retraining

D. Database Session Management

FastAPI dependencies yield exactly one asynchronous session per incoming request, automatically managing commit and rollback transaction boundaries.

```
async def get_db():
    async with AsyncSessionLocal() as session:
        yield session
```

Listing 3: Per-request async database session dependency

E. Asyncpg Password Handling

The `asyncpg` driver raises connection errors when it encounters unescaped special characters (such as `#` or `@`) inside the connection URL's password segment. This is mitigated with the `safe_async_db_url` property exposed by `app.config.Settings` (see Section VIII), which URL-encodes the raw password before substitution.

TABLE VIII: Authentication Endpoints

Endpoint	Method	Auth	Purpose
/api/v1/auth/register	POST	No	Creates a new user account
/api/v1/auth/login	POST	No	Authenticates user, returns JWT pair
/api/v1/auth/refresh	POST	Refresh token	Issues new Access Token
/api/v1/auth/me	GET	Yes	Fetches active user profile
/api/v1/auth/logout	POST	Yes	Invalidates active session

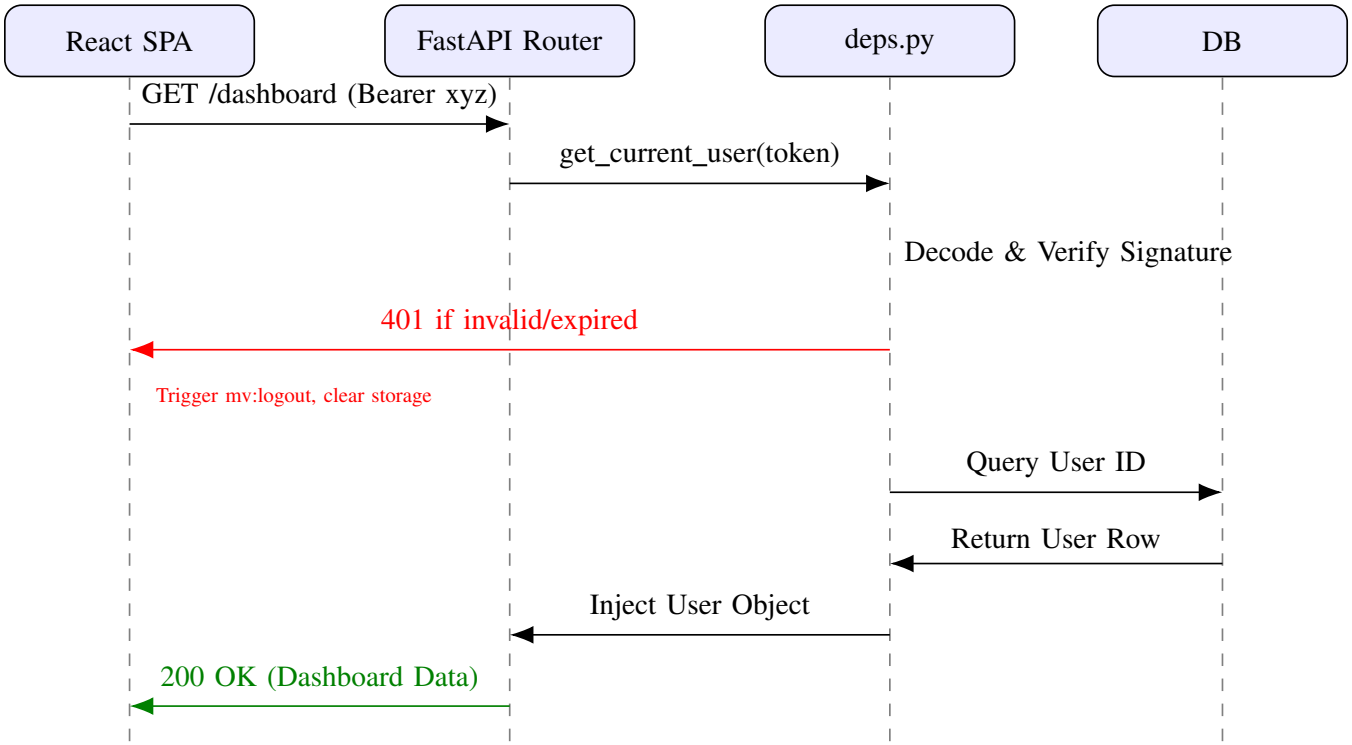

Fig. 4: JWT-protected request sequence diagram.

F. Key Indexing Strategy

G. Alembic Migration Workflow

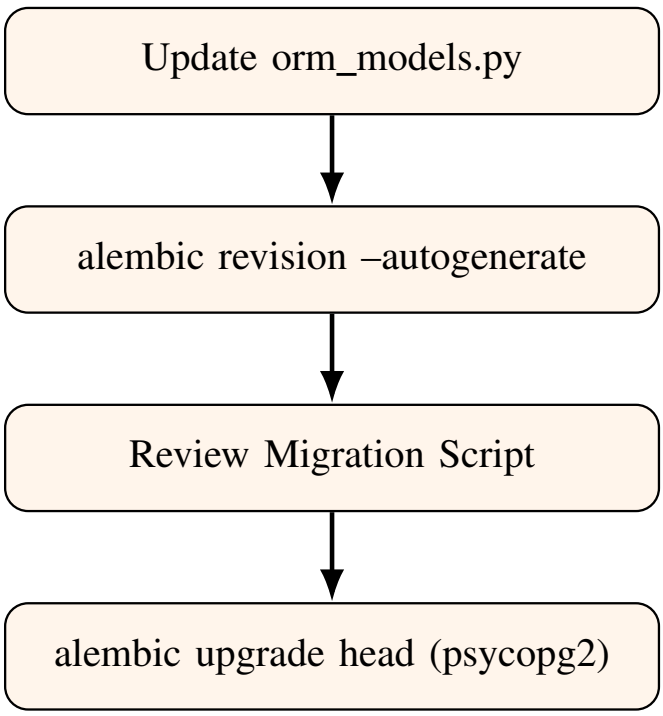

Fig. 5: Alembic schema migration workflow.

VIII. Configuration Design (pydantic-settings)

A. Single Source of Truth Pattern

Movientum enforces a strict Single Source of Truth configuration pattern using `pydantic-settings`. All environment variables are parsed and type-validated at application startup; the application refuses to boot if any critical configuration value — such as a database URL or API key — is missing or malformed. Direct use of `os.getenv()` is explicitly forbidden anywhere outside the configuration layer itself, guaranteeing that every consumer of configuration reads from one validated, cached object.

B. The Settings Class

```python
from pydantic_settings import BaseSettings
from pydantic import field_validator
from functools import lru_cache

class Settings(BaseSettings):
    # TMDB
    tmdb_api_key: str
    tmdb_read_access_token: str
    tmdb_base_url: str = "https://api.themoviedb.org/3"

    # Database
    database_url: str           # sync (psycopg2) for
     Alembic
    async_database_url: str     # asyncpg for FastAPI
```

TABLE X: Key Indexing Strategies

Table	Index	Purpose
movies	idx_movies_popularity (DESC)	Fast retrieval of trending items
movies	idx_movies_fts (GIN)	Full-Text Search over TSVECTOR
interaction_log	idx_interaction_log_user_ts	Fast retrieval for 30-day nightly retrain query
content_catalog	uq_catalog_tmdb_media	Prevents ingestion duplicates for movies/tv

```
    db_password: str

    # Redis & Celery
    redis_url: str
    celery_broker_url: str = ""
    celery_result_backend: str = ""

    # JWT
    jwt_secret_key: str
    jwt_algorithm: str = "HS256"
```

Listing 4: Core Settings class (backend/app/config.py)

C. Custom Validators and URL Cleaning

Upstash Redis connection URLs frequently embed query parameters (such as `ssl_cert_reqs`) that are incompatible with the Celery broker parser or with standard Redis client parsers. A `field_validator` hook strips these problematic query parameters at startup time before the URL is used anywhere downstream.

```
@field_validator("redis_url", mode="before")
@classmethod
def clean_redis_url(cls, v):
    if v and "?" in v:
        base, query = v.split("?", 1)
        params = [p for p in query.split("&")
                  if not p.lower().startswith("
     ssl_cert_reqs=")]
        v = f"{base}?{'&'.join(params)}" if params else
     base
    return v
```

Listing 5: Redis URL cleaning validator

D. Password Encoding Proxies

Because `asyncpg` fails to parse connection URLs whose password segment contains URL-unsafe characters (`#`, `@`), the Settings class exposes a computed property that securely injects a URL-encoded copy of the password:

```
@property
def safe_async_db_url(self) -> str:
    from urllib.parse import quote_plus
    encoded = quote_plus(self.db_password)
    return self.async_database_url.replace(self.
     db_password, encoded)
```

Listing 6: Safe URL-encoded database URL property

E. Singleton Instantiation

To avoid re-parsing the `.env` file on every single request, the settings object is memoized using `@lru_cache()`, and all other modules simply import the ready-made `settings` singleton from `app.config`.

```
@lru_cache()
def get_settings() -> Settings:
    return Settings()

settings = get_settings()
```

Listing 7: Cached settings singleton

F. Environment Variable Reference

G. Configuration Load Workflow

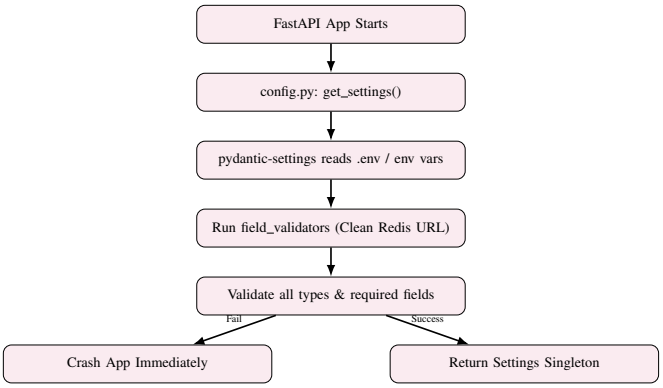

Fig. 6: Configuration load and validation workflow.

IX. Ratings System

A. Overview and Architecture

Movientum deliberately rejects the traditional 5-star or 10-point rating scale. In its place is a proprietary, four-category categorical rating system designed to capture the user's *emotional intent* and *viewing context* rather than an arbitrary numerical score. Text reviews are explicitly **not** supported anywhere on the platform — ratings are the sole qualitative signal, keeping the interaction model fast, low-friction, and structured for direct machine-learning consumption.

B. The Four Categories

C. Machine Learning Impact

These categorical ratings serve as the strongest explicit signals feeding the Recommendation Engine (Section XII). A `perfection` rating aggressively updates the `UserTasteProfile` JSONB vectors across Genre, Cast, and Crew dimensions, heavily boosting those traits, while a `skip` rating applies a negative weight penalty that teaches the underlying XGBoost model to actively avoid structurally similar content in future ranking passes.

D. Database Implementation

Ratings are persisted in the `ratings` table, with the `rating` column constrained to integer values 1 through 4. A unique constraint on `(user_id, movie_id)` ensures each user maintains exactly one active rating per title, implementing upsert semantics rather than an append-only log.

```
class Rating(Base):
    __tablename__ = "ratings"
    id = Column(Integer, primary_key=True)
    user_id = Column(UUID(as_uuid=True), ForeignKey("users
     .id"))
```

TABLE XI: Required and Optional Environment Variables

Variable	Required	Description
tmdb_api_key / tmdb_read_access_token	Yes	Authentication for TMDB API ingestion
database_url	Yes	Synchronous DB URL (psycopg2) for Alembic
async_database_url	Yes	Asynchronous DB URL (asyncpg) for runtime
db_password	Yes	Raw DB password, used for URL-encoding injection
redis_url	Yes	Upstash Redis connection URL (tls: rediss://)
jwt_secret_key	Yes	Secret used to sign auth tokens
APPLICATIONINSIGHTS_CONNECTION_STRING	No	OpenTelemetry hook (parsed in telemetry.py)

TABLE XII: Rating Category Definitions

Category	Value	Semantic Meaning
skip	1	"I regret watching this / Do not recommend."
timepass	2	"It was okay to have on in the background, but nothing special."
go_for_it	3	"Solid movie, highly recommend for a movie night."
perfection	4	"Absolute masterpiece, must-watch."

```
    movie_id = Column(Integer, ForeignKey("movies.id"))
    rating = Column(Integer, nullable=False)
    # 1=skip, 2=timepass, 3=go_for_it, 4=perfection
    created_at = Column(DateTime(timezone=True),
     server_default=func.now())
```

Listing 8: Rating ORM model

E. Routing Surface

F. ML Signal Weight Table

G. Rating Submission Workflow

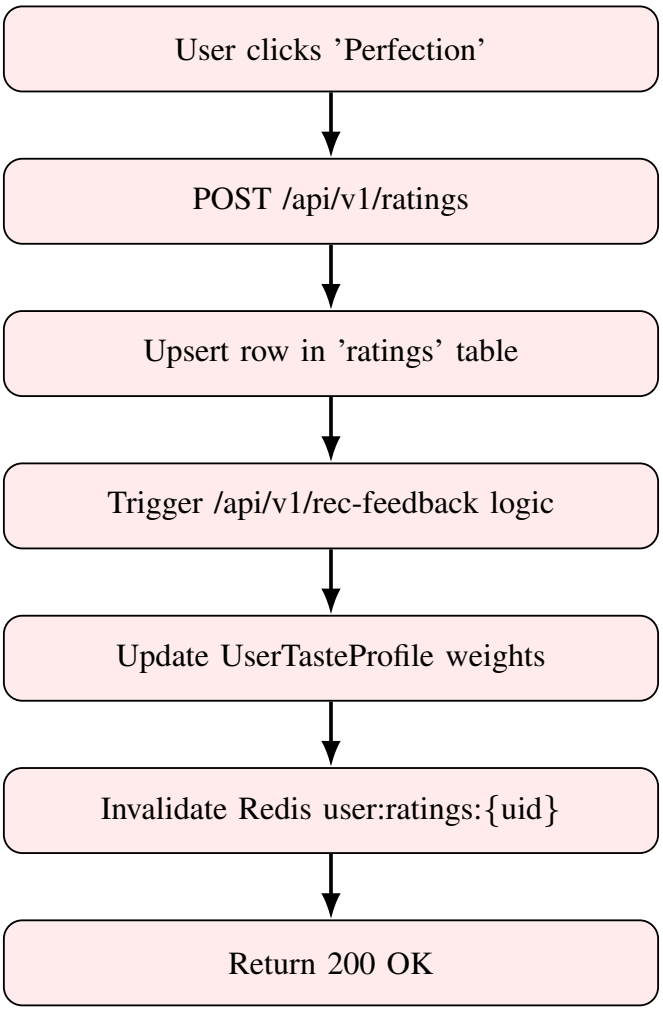

Fig. 7: Rating submission and downstream ML update workflow.

TABLE XIII: Ratings API Endpoints

Endpoint	Method	Purpose
/api/v1/ratings	POST	Upserts a user's rating for a specific movie_id
/api/v1/ratings	DELETE	Removes a rating
/api/v1/ratings/me	GET	Fetches all ratings for the authenticated user (Dashboard)

TABLE XIV: Rating Category to Taste-Profile Impact Mapping

Rating Category	Integer	Taste Profile Impact
perfection	4	Huge Positive (+3.0)
go_for_it	3	High Positive (+1.5)
timepass	2	Mild Positive (+0.5)
skip	1	High Negative (-2.0)

X. Search and Filtering

A. Overview and Architecture

Movientum's search infrastructure relies purely on PostgreSQL's built-in Full-Text Search (`TSVECTOR`) and Trigram (`pg_trgm`) extensions. This design choice avoids the operational complexity of standing up and maintaining an external Elasticsearch or Meilisearch cluster, while still delivering rapid, typo-tolerant search across tens of thousands of catalog records.

B. Two-Tier Search System

1) **Instant Search (Autocomplete)**: Fires on every keystroke (debounced client-side). Searches only the `title` field using `pg_trgm`'s `%` similarity operator for fast prefix and fuzzy matches. Results are cached in Redis under `search:auto:{prefix}` with a TTL of 300 seconds.
2) **Paginated Deep Search**: Fires when the user presses Enter or visits the `/explore` page. Uses the `search_vector` (`TSVECTOR`) column to search across titles, overviews, and actor names simultaneously, supporting pagination and genre filtering. Results are cached under `search:paged:{hash}` with a TTL of 600 seconds.

C. Database Implementation

A dedicated periodic script, `ingest_search_index.py`, maintains the `search_vector` column by concatenating the movie title (Weight A) and overview (Weight B):

```
-- Executed by ingest_search_index.py
UPDATE movies
SET search_vector =
    setweight(to_tsvector('english', coalesce(title, '')),
      'A') ||
    setweight(to_tsvector('english', coalesce(overview, ''
    )), 'B');

CREATE INDEX idx_movies_fts ON movies USING GIN(
    search_vector);
CREATE INDEX idx_movies_title_trgm ON movies USING GIST(
    title gist_trgm_ops);
```

Listing 9: Full-text search vector maintenance and indexing

Weighting titles as Weight A and overviews as Weight B ensures that a query term matching the title ranks a result more highly than the same term merely appearing somewhere within a lengthy plot synopsis, which materially improves perceived search relevance without any additional ranking logic.

D. The Search Router

TABLE XV: Search API Endpoints

Endpoint	Description
GET /api/v1/search/autocomplete	Fast path: checks Redis, then Postgres via ILIKE + trigram similarity
GET /api/v1/search/paged	Deep path: accepts query, page, type (movie/tv/person), genre

E. Filter Capability Matrix

F. Full-Text Search Flow

XI. Watchlist and Collections Feature

A. Overview and Architecture

Movientum implements a multi-collection watchlist system rather than a single binary "Watchlist" flag. Users may create unlimited custom named collections (e.g. "Sci-Fi Weekend", "Movies to watch with Mom") and add movies or TV shows to any of them. This design is implemented in `watchlist.py` and its corresponding `watchlist_repo.py` repository.

B. Cross-Media Support

A single collection may freely contain a mixture of Movies and TV Shows. This polymorphism is handled gracefully at the schema level via a `media_type` discriminator column (`'movie'` or `'tv'`), avoiding the need for two entirely separate collection tables.

C. Cache Aggressiveness Rationale

Because checking whether a given item is already present in a user's watchlist is required on almost every single movie-card render throughout the UI (to determine whether the bookmark icon should render filled or unfilled), the `GET /api/v1/watchlists/movie/{media_type}/{movie_id}/st` endpoint must respond with extremely low latency. Consequently, whenever a user adds or removes an item, the backend issues an aggressive Redis invalidation that busts both the specific collection's cache entry and the user's master collection-list cache entry in a single operation (`_bust_collection`).

TABLE XVI: Explore Page Filter Capabilities

Filter Type	Backend Implementation	Performance
Text Query	search_vector @@ to_tsquery	High (GIN Index)
Genre	INNER JOIN movie_genres	High (B-Tree Index)
Media Type	Union Queries (Movies + TV)	Moderate
Sort By	ORDER BY popularity DESC	High (B-Tree Index)

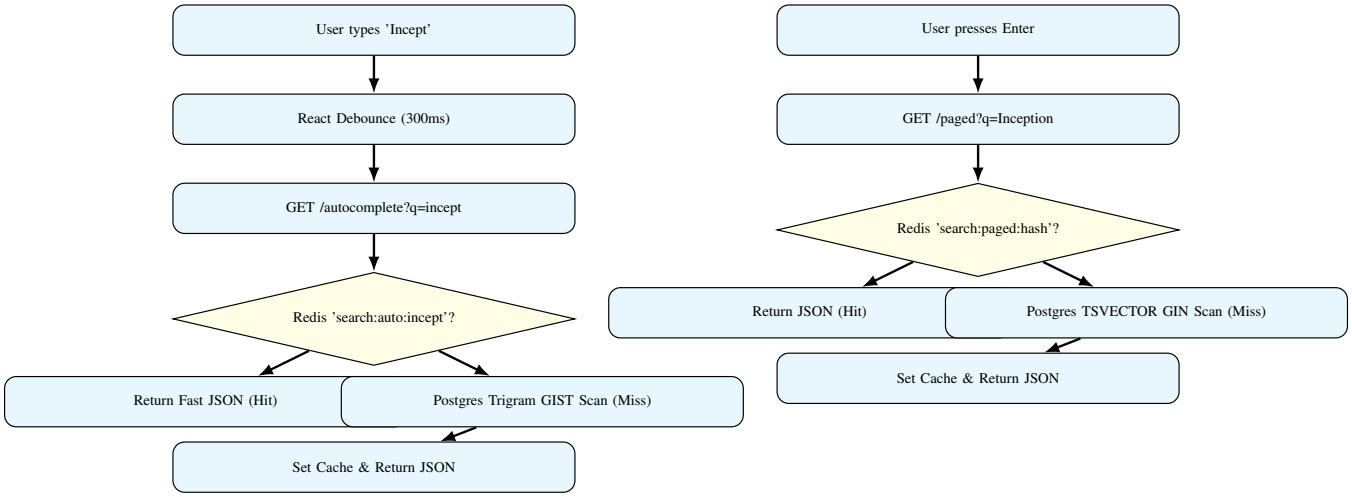

Fig. 8: Autocomplete (instant) and paginated (deep) search flows.

D. Database Schema

```python
class WatchlistCollection(Base):
    __tablename__ = "watchlist_collections"
    id = Column(UUID(as_uuid=True), primary_key=True,
     default=uuid.uuid4)
    user_id = Column(UUID(as_uuid=True), ForeignKey("users
     .id"))
    name = Column(String(100), nullable=False)
    description = Column(Text, nullable=True)

class WatchlistItem(Base):
    __tablename__ = "watchlist_items"
    id = Column(Integer, primary_key=True)
    collection_id = Column(UUID(as_uuid=True),
                           ForeignKey("
     watchlist_collections.id"))
    movie_id = Column(Integer, nullable=False)
    # Maps to either movies.id or tv_shows.id
    media_type = Column(String(20), nullable=False,
     default="movie")
```

Listing 10: Watchlist collection and item ORM models

E. Endpoint Reference

TABLE XVII: Watchlist API Endpoints

Endpoint	Description
GET /api/v1/watchlists	Lists all collections for the user
POST /api/v1/watchlists	Creates a new named collection
GET /api/v1/watchlists/{collection_id}	Retrieves a specific collection with hydrated items
POST /api/v1/watchlists/{collection_id}/items	Adds an item
DELETE /api/v1/watchlists/{collection_id}/items/{media_type}/{id}	Removes an item

TABLE XVIII: Watchlist Redis Cache Keys

Cache Key Pattern	Bust Trigger	TTL
user:wlists:{user_id}	On Create/Delete Collection	300s (5m)
user:wlcoll:{collection_id}	On Add/Remove Item	300s (5m)

F. Redis Key Table

G. Add-to-Watchlist Workflow

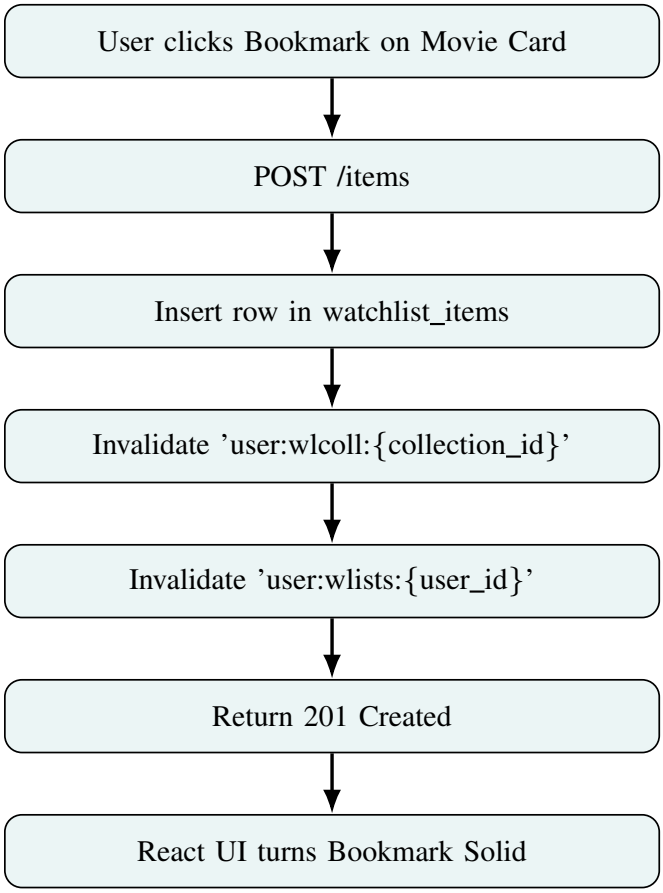

Fig. 9: Add-to-watchlist end-to-end workflow.

XII. Recommendation Engine: Complete Overview

A. System Overview

Movientum's advanced recommendation engine is a full ML-grade personalized system running on top of FastAPI, SQLAlchemy, Supabase PostgreSQL, Upstash Redis, Celery, and the React SPA frontend. The engine is organized as a strict six-tier pipeline, described in full in this and the following sections.

B. The Six-Tier Pipeline

1) **Tier 1 — Data Foundation**: the fast, local `content_catalog` table and the live `user_taste_profiles` table.
2) **Tier 2 — On-Demand Feature Ingestion**: cache-on-demand TMDB fallback ingestion with database backfill — any new TMDB item is automatically ingested into the catalog on its very first query.
3) **Tier 3 — Graph Candidate Retrieval**: a bipartite Content Graph enabling Personalized PageRank via Random Walk with Restart (RWR).
4) **Tier 4 — Feature Matrix and Inference**: an `XGBRanker` prediction step using static, personalized, and structural-overlap features.
5) **Tier 5 — Ensemble Blending**: Team-Draft Interleaving (TDI), blending the new ML model's output with an existing baseline popularity model.
6) **Tier 6 — Feedback and Retraining**: real-time user interaction signals (including watch-history parsing) feed a nightly, fully-automated model retrain executed via Celery Beat.

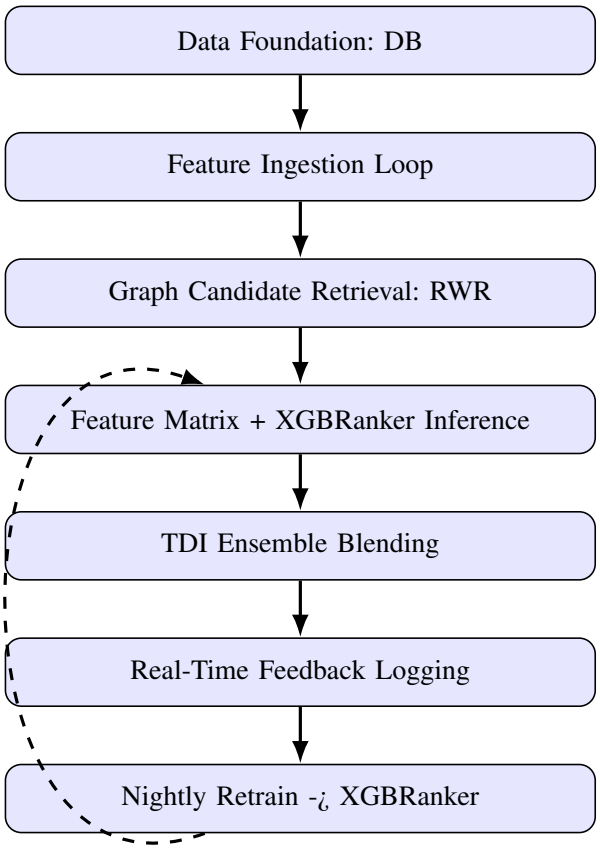

Fig. 10: Recommendation engine six-tier architecture flow, with the retraining feedback loop closing back into the inference stage.

C. Graph Construction and Edge Weights

The bipartite content graph contains six node types: Content, Genre, Keyword, Talent (cast/crew), Era, and Language.

Edge weights are hand-tuned to reflect their empirical predictive importance for taste similarity:

The ordering in Table XIX encodes a specific editorial hypothesis: a director's stylistic signature (mise-en-scène, pacing, thematic preoccupation) is the single strongest predictor that a viewer who liked one of their films will like another, more so than shared cast members (actors work across wildly different genres and directors) or shared genre labels alone (which are coarse and often overly broad, e.g. "Drama").

D. Interaction Signal Scoring

Explicit signals (thumbs up/down) are given zero time decay ($\lambda = 0$), meaning their influence on the taste profile never diminishes with time, whereas implicit signals (poster clicks, watches) decay exponentially with $\lambda = 0.01$, corresponding to a half-life of approximately 69 days. All profile weights are clamped to the range $[-100, 100]$ to prevent runaway accumulation from power users with very high interaction volume.

Note that the README's simplified public-facing signal table (Table XXI below) presents a condensed subset of this same mechanism for onboarding purposes; Table XX above is the authoritative, complete Phase 6 specification.

E. Taste Profile Rebuilding and Cold Start

The backend supports fully rebuilding a `UserTasteProfile` from scratch using a user's historical `WatchHistory` and `Watchlist` rows via `rebuild_taste_profile_from_history`, guaranteeing accurate cold-start personalization even for users who join after having already interacted with the catalog through an import mechanism. For genuinely new users with zero interactions, the system falls back to baseline popularity ranking blended with genre-onboarding weights until sufficient real signal accumulates.

F. Data Model: ContentCatalog

```
tmdb_id          = Column(Integer, nullable=False)
media_type       = Column(String(10), nullable=False)
genre_ids        = Column(ARRAY(Integer), default=[])
keyword_ids      = Column(ARRAY(Integer), default=[])
cast_ids         = Column(ARRAY(Integer), default=[])
crew_ids         = Column(JSONB, default={})
# {"director": [id], "writer": [id]}
original_language = Column(String(10))
release_era       = Column(String(20))
vote_average     = Column(Float, default=0.0)
popularity       = Column(Float, default=0.0)
```

Listing 11: ContentCatalog ORM model — fast-inference structure

G. Data Model: UserTasteProfile

```
genre_weights    = Column(JSONB, default={})
cast_weights     = Column(JSONB, default={})
crew_weights     = Column(JSONB, default={})
keyword_weights  = Column(JSONB, default={})
language_weights = Column(JSONB, default={})
era_weights      = Column(JSONB, default={})
negative_weights = Column(JSONB, default={})
# Populated by thumbs_down & unwatched
```

Listing 12: UserTasteProfile ORM model — six JSONB weight vectors

Each weight dictionary maps a TMDB entity ID (as a string key, since JSONB does not support integer keys) to a clamped floating-point affinity score. At inference time, these six vectors are looked up against each candidate item's own genre/cast/crew/keyword/language/era arrays to compute the six `user_*_score` features described in Section XIII.

H. Data Model: InteractionLog

```
user_id          = Column(UUID(as_uuid=True), ForeignKey("
    users.id"))
tmdb_id          = Column(Integer, nullable=False)
media_type       = Column(String(10), nullable=False)
signal_type      = Column(String(20), nullable=False)
label            = Column(Integer, nullable=False)
feature_snapshot = Column(JSONB, default={})
# The 16-dim feature vector at time of interaction
timestamp        = Column(DateTime(timezone=True), default
    =utcnow)
```

Listing 13: InteractionLog ORM model — nightly training dataset source

Storing the *exact* 16-dimensional feature vector that was in effect at the moment of interaction (rather than recomputing it later at training time) is a deliberate correctness decision: features such as `ppr_score` and `user_genre_score` are highly time-dependent — the user's taste profile and the item's graph proximity may both have shifted substantially by the time the nightly retrain runs. Snapshotting guarantees that the training label is always paired with the feature values that actually produced the ranking the user reacted to.

I. Inference Pipeline Steps

The core inference pipeline, implemented in `advanced_recs.py`, executes the following steps for every recommendation request:

1) Ensure the seed item exists in the catalog (via `ingest_item_to_catalog`), triggering on-demand TMDB ingestion if it is missing.
2) Load the cached `networkx.Graph` singleton from `graph_cache.py`.
3) Run `get_rwr_candidates` to retrieve candidate node IDs via Random Walk with Restart.
4) Construct a 16-feature `numpy.ndarray` matrix matching each candidate against the user's `UserTasteProfile`.
5) Run `rank_candidates(feature_matrix)` using the `XGBRanker` wrapper in `ranker.py`. If the model is not yet trained (cold start), fall back to sorting purely by `ppr_score` (feature column 0).

XIII. Feature Matrix, XGBRanker, and Mathematical Formalization

A. The 16-Feature Matrix

Every candidate item entering the ranking stage is represented by a 16-dimensional feature vector, grouped into four semantic families:

The feature set is intentionally partitioned into: (a) *graph-derived* features that capture structural proximity within the bipartite content graph independent of any single user, (b) *content-derived* features that capture the item's own static

TABLE XIX: Graph Edge Weights and Rationale

Feature	Edge Weight	Rationale
Director	2.5	Strongest creative signal
Keyword	1.5	Thematic specificity beyond genres
Genre	1.0	Baseline similarity
Cast	0.8	Moderate talent signal
Era	0.6	Decade-taste preference
Language	0.4	Weakest but still useful

TABLE XX: Interaction Signal Types, Labels, and Profile Deltas

Signal	Label	Profile Delta
Thumbs Up	3	genres +10.0, cast +10.0, crew +10.0, era +10.0, keyword +5.0, language +1.0
Thumbs Down	-1	genres -15.0, cast -15.0, crew -15.0, era -15.0, keyword -8.0, language -1.5
Poster Click (decaying)	2	genres +2.0, keyword +1.0, language +0.2
Watched	4	genres +15.0, cast +10.0, crew +10.0, era +10.0, keyword +8.0, language +2.0
Unwatched	0	genres -15.0, cast -10.0, crew -10.0, era -10.0, keyword -8.0, language -2.0
Watchlist Add	3	genres +8.0, cast +5.0, crew +5.0, era +5.0, keyword +4.0, language +1.0
Watchlist Remove	0	genres -8.0, cast -5.0, crew -5.0, era -5.0, keyword -4.0, language -1.0

TABLE XXI: Simplified Public Signal Summary (README)

Signal	Profile Delta	Notes
Thumbs Up	+10.0 on genres/cast/crew/era	Explicit — no decay
Poster Click	+2.0 on genres	Implicit, decay applied
Scroll Ignore (2s view, no click)	-0.5 on genres	Implicit negative signal
Thumbs Down	-15.0 on genres/cast/crew/era	Explicit rejection — no decay

TABLE XXII: Complete 16-Feature Matrix Column Reference

Feature Name	Source	Description
ppr_score	Graph RWR	Network visit probability from the Personalized PageRank walk
ppr_rank_norm	Graph RWR	Normalized rank position of the PPR score among all candidates
vote_average	Catalog	Static TMDB average vote score
vote_count_log	Catalog	Log-scaled TMDB vote count (dampens popularity outliers)
popularity_log	Catalog	Log-scaled TMDB popularity index
recency_score	Catalog	Score favoring more recently released content
user_genre_score	Taste Profile	Personalized genre affinity from UserTasteProfile
user_cast_score	Taste Profile	Personalized cast affinity
user_crew_score	Taste Profile	Personalized crew (director/writer) affinity
user_keyword_score	Taste Profile	Personalized thematic keyword affinity
user_era_score	Taste Profile	Personalized decade/era affinity
user_language_mult	Taste Profile	Personalized language multiplier (1.0 = neutral)
genre_overlap_count	Catalog	Count of genres shared with the seed/origin item
cast_overlap_count	Catalog	Count of cast members shared with the seed/origin item
same_language	Catalog	Binary flag: same original language as seed item
same_era	Catalog	Binary flag: same release era/decade as seed item

quality and popularity signals, (c) *personalized* features that capture this specific user's affinity for the item's attributes, and (d) *structural overlap* features that directly measure similarity to whatever seed item triggered the recommendation request. This four-way decomposition allows the gradient-boosted tree ensemble to learn interaction effects between global popularity, personal taste, and structural similarity without requiring any manual feature-crossing.

B. Random Walk with Restart — Mathematical Definition

Given the bipartite content graph $G = (V, E)$ with weighted edges as in Table XIX, and a seed node $s \in V$, the Personalized PageRank vector π is the stationary solution of:

$$\pi = (1 - \alpha)\, W\pi + \alpha\, e_s \tag{1}$$

where W is the column-stochastic transition matrix derived from the weighted adjacency structure of G, $\alpha \in (0, 1)$ is the restart probability (the probability at each step of teleporting back to the seed node s rather than continuing the random walk), and e_s is the standard basis vector with a 1 in position s and 0 elsewhere. The resulting π_v for each node v is interpreted as the long-run probability of a random walker, who periodically resets to s, currently being located at v. Nodes with high π_v are structurally close to the seed item through short, high-weight paths (e.g. sharing the same director, several keywords, or genre) and are retained as the top-K (typically $K = 80$–100) candidates for downstream ranking.

C. Why RWR Over Plain Cosine Similarity

A naive alternative would compute cosine similarity between item feature vectors directly. RWR is preferred because it captures *multi-hop* transitive similarity — e.g. two movies sharing no genre or cast member directly, but both connected strongly to the same keyword node and the same director node, will still receive elevated π_v mass through the graph's intermediate nodes, whereas a flat cosine-similarity computation over raw feature vectors would miss this indirect relationship entirely.

D. XGBRanker Objective

The re-ranking model is an `XGBRanker` trained with the pairwise/listwise `rank:ndcg` objective, which directly optimizes Normalized Discounted Cumulative Gain at the top of the ranked list:

$$\text{NDCG@}k = \frac{\text{DCG@}k}{\text{IDCG@}k}, \qquad \text{DCG@}k = \sum_{i=1}^{k} \frac{2^{rel_i} - 1}{\log_2(i+1)} \tag{2}$$

where rel_i is the graded relevance label (drawn from the interaction-log label space $\{-1, 0, 2, 3, 4\}$ described in Section XV) of the item ranked at position i, and IDCG@k is the DCG of the ideal (perfectly sorted) ranking, used to normalize the score into $[0, 1]$. Optimizing NDCG directly — rather than a simple regression or classification loss — ensures the model is explicitly rewarded for placing the most relevant items as close to the top of the recommendation feed as possible, which is precisely the user-facing quantity that matters, since most users only scroll through the first handful of recommended items.

E. Cold-Start Fallback

If `is_model_trained()` reports that no XGBRanker model has yet been persisted to `ranker.json` (e.g. on a brand-new deployment prior to the first nightly retrain), `rank_candidates` falls back to sorting candidates purely by column 0 of the feature matrix — the raw `ppr_score` — ensuring the system always returns a coherent, graph-grounded ranking even before any supervised model exists.

XIV. Content-Based Recommendations (Basket / "Find Similar")

A. Goal

The `/rec-content` feature allows a user to build a basket of several movies or TV shows and request a "Find Similar" expansion. The backend extracts common traits across the basket, combines an RWR graph query with a live TMDB Discovery API call, interleaves the two candidate pools via Team-Draft Interleaving, and scores the result for final relevance ranking.

B. Deterministic Seeding and Pagination Caching

To keep pagination stable across infinite-scroll sessions without ever serving a duplicate item, the incoming basket is canonically sorted and hashed, and that hash seeds Python's random-number generator deterministically for the session:

```
# Deterministic Seeding
sorted_items_json = json.dumps(
    sorted(items, key=lambda x: (x["tmdb_id"], x["
     media_type"])))
items_hash = hashlib.sha256(sorted_items_json.encode()).
     hexdigest()
random.seed(items_hash)

# Page Cache Check
page_cache_key = f"rec:content:{items_hash}:page:{page}"
cached_page = await get_cached(page_cache_key)
if cached_page:
    return cached_page
```

Listing 14: Deterministic basket hashing and page cache check

C. Trait Extraction

The backend fetches the `ContentCatalog` rows for every basket item and extracts: `basket_genres` (union of all genres across the basket), `basket_keywords` (union of all keywords), `common_genres` (strict intersection of genres across *all* basket items), and `top_seeds` (the two most popular items in the basket, used as graph traversal seeds).

D. Strategy A: Graph Random Walk with Restart

```
G = await get_or_build_graph(db)
merged_rwr_score = {}
if G:
    for seed_row in top_seeds:
        seed_node = f"{seed_row.media_type}_{seed_row.
     tmdb_id}"
        if G.has_node(seed_node):
            candidates = get_rwr_candidates(G, seed_node,
     top_k=80)
            for k, v in candidates.items():
                merged_rwr_score[k] = max(merged_rwr_score
     .get(k, 0), v)
```

Listing 15: RWR candidate merging across multiple basket seeds

Merging via element-wise `max` rather than sum or average across the two seed walks ensures that an item strongly connected to *either* basket seed is surfaced, without artificially inflating (or double-counting) items that happen to be moderately connected to both.

E. Strategy B: TMDB Discovery

When `common_genres` exist (or, failing that, up to three genres are sampled from `basket_genres`), the system augments the graph candidates with fresh live data pulled directly from TMDB's Discover API:

```
discover_genres = list(common_genres) if common_genres \
    else list(basket_genres)[:3]
if discover_genres:
    genre_str = ",".join(map(str, discover_genres))
    tmdb_page = math.ceil(page / 2.0) if page > 0 else 1
    movies = await tmdb_service_instance.discover_movies(
        genre_str, page=tmdb_page)
```

Listing 16: TMDB Discovery augmentation

F. Blending via Team-Draft Interleaving

Strategy A and Strategy B candidate pools are blended using Team-Draft Interleaving (TDI) with a 70% draft preference toward the graph-derived (RWR) pool:

```
blended = team_draft_interleave(
    rwr_candidates,
    tmdb_candidates,
    k=len(rwr_candidates) + len(tmdb_candidates),
    ratio_a=0.7
)
```

Listing 17: Team-Draft Interleaving invocation

Team-Draft Interleaving is a classic technique from information-retrieval evaluation, here repurposed as a production blending mechanism: at each draft "pick", a biased coin (weighted `ratio_a`) determines whether the next slot is filled from pool A or pool B, with already-drafted items skipped, guaranteeing a fair, deduplicated, ratio-respecting merge of the two ranked lists.

G. Hard Filters Applied

Four hard filters are applied to the blended candidate stream before scoring:

1) Items already present in the user's basket are excluded.
2) Items with `vote_count` < 50 are excluded (removes statistically unreliable, low-sample-size ratings).
3) Items with no `poster_path` are excluded (prevents broken UI cards).
4) Items already served on an earlier paginated page are excluded, tracked via the `rec:content:{items_hash}:seen` Redis set.

H. Final Scoring Function

Each surviving candidate receives a final weighted composite score:

$$\begin{aligned}\text{final\_score} = {} & 0.5 \cdot \text{genre\_score} + 0.2 \cdot \text{keyword\_score} \\ & + 0.15 \cdot \text{rating\_score} + 0.1 \cdot \text{pop\_score} \\ & + 0.2 \cdot \text{normalized\_rwr\_score}\end{aligned} \tag{3}$$

where $\text{genre\_score} = |c_{\text{genre}} \cap \text{basket\_genres}|$, $\text{keyword\_score} = |c_{\text{keyword}} \cap \text{basket\_keywords}|$, $\text{rating\_score} = \text{vote\_average}/10$, $\text{pop\_score} = \ln(1 + \text{popularity})$, and normalized_rwr_score is the candidate's merged RWR score divided by the maximum RWR score observed in the candidate pool. The dominant weight of 0.5 on genre overlap reflects that, for an explicit multi-item basket (as opposed to a single-seed recommendation), shared genre identity across the whole basket is the most reliable indicator of what the user is collectively looking for, while the smaller RWR and popularity terms provide a secondary tie-breaking signal informed by deeper graph structure and general quality.

The top 20 scored candidates per page are returned to the client, and the `seen` cache set is updated to guarantee no duplicate is ever served on a subsequent page of the same basket session.

I. Edge Cases

If both Strategy A and Strategy B yield zero candidates, the endpoint returns an empty list rather than erroring. Deduplication within `team_draft_interleave` keys on the composite `(tmdb_id, media_type)` tuple, guaranteeing no duplicate item can ever appear twice within a single interleaved pool.

J. Cache Key Reference

K. Content-Based Recommendation Flow

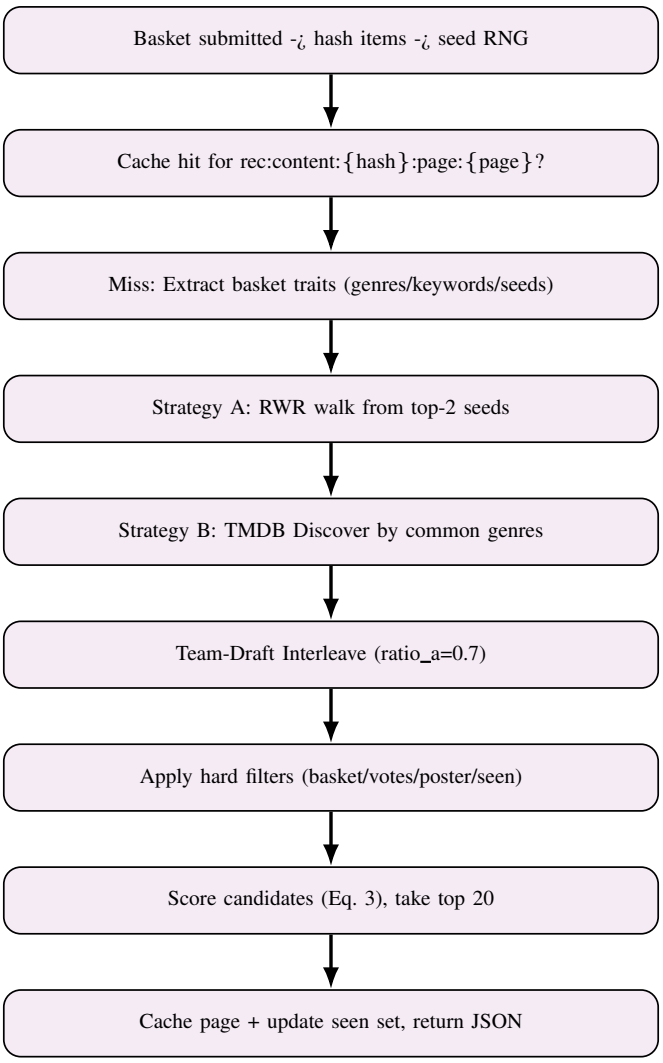

Fig. 11: End-to-end content-based (basket) recommendation flow.

XV. Nightly XGBRanker Retraining Pipeline

A. Goal and Schedule

A Celery Beat task executes nightly at 3:30 AM IST to rebuild the `XGBRanker` recommendation model from user interactions collected over the trailing 30-day window. The resulting model artifact is saved locally to `ranker.json`, hot-reloaded into the live inference process, and the graph cache is invalidated so that the very next user request rebuilds

TABLE XXIII: Content-Based Recommendation Cache Keys

Cache Key	TTL	Description
rec:content:{hash}:page:{page}	900s (15m)	Fully formatted JSON response for the specific page
rec:content:rwr:{hash}	1800s (30m)	Cached basket_genres, basket_keywords, common_genres, merged_rwr_score
rec:content:{hash}:seen	1800s (30m)	Set of (tmdb_id, media_type) already served on previous pages

the bipartite graph from the freshest catalog state. Note: MLflow tracking was evaluated and explicitly bypassed/removed in favor of direct disk artifact saves plus OpenTelemetry metrics, in order to minimize infrastructure overhead for a system of this scale.

B. Training Data Assembly

The pipeline, implemented in `backend/app/ml/training.py`, begins by pulling all `InteractionLog` rows from the past `_LOOKBACK_DAYS = 30` days:

```
cutoff = datetime.now(timezone.utc) - timedelta(days=
    _LOOKBACK_DAYS)
result = await db.execute(
    select(InteractionLog)
    .where(InteractionLog.timestamp >= cutoff)
    .order_by(InteractionLog.user_id, InteractionLog.
    timestamp)
)
logs = result.scalars().all()
```

Listing 18: 30-day interaction log retrieval for training

Three additional steps complete the training-set assembly:

- **Label Generation**: the explicit raw labels stored on each log row (`log.label` $\in \{-1, 0, 2, 3, 4\}$, per Table XX) are used directly as ranking targets.
- **Sample Weighting (Time Decay)**: an exponential time-decay weight, `decay = time_decay_weight(log.timestamp)`, down-weights older interactions relative to more recent ones within the 30-day window.
- **Feature Extraction**: the pre-snapshotted 16-dimensional `feature_snapshot` attached to each log row is used directly to construct the feature matrix X, avoiding any risk of feature/label temporal mismatch (see Section XII).

Rows are additionally grouped by `user_id` so that `XGBRanker` can optimize ranking quality *within* each individual user's session rather than across unrelated users' interaction streams, which is the correct grouping semantics for a listwise/pairwise ranking loss such as `rank:ndcg`.

C. Train/Validation Split

The dataset is split such that the last 15% of user groups (`_VAL_FRACTION = 0.15`) are strictly held out for validation, with the split boundary chosen to preserve intact ranking query groups (i.e. a single user's interaction sequence is never split across both the train and validation partitions):

```
n_groups = len(groups)
val_groups_count = max(1, int(n_groups * _VAL_FRACTION))
train_groups_count = n_groups - val_groups_count
```

Listing 19: Group-preserving train/validation split

D. XGBRanker Training Configuration

The model is trained with the `rank:ndcg` objective (Equation 2) to directly optimize top-k ranking quality, with early stopping enabled against a held-out validation NDCG@10 metric:

```
ranker = XGBRanker(
    objective        = "rank:ndcg",
    n_estimators     = 300,
    max_depth        = 6,
    learning_rate    = 0.05,
    subsample        = 0.8,
    colsample_bytree = 0.8,
    tree_method      = "hist",
    eval_metric      = "ndcg@10",
    early_stopping_rounds = 20,
    verbosity        = 0,
)

ranker.fit(
    X_train, y_train,
    group = train_groups,
    sample_weight = group_weights,
    eval_set = [(X_val, y_val)],
    eval_group = [val_groups],
    verbose = False,
)
```

Listing 20: XGBRanker hyperparameter configuration and fit call

1) Hyperparameter Rationale

`tree_method="hist"` is chosen specifically for CPU-bound inference efficiency in the production serving path (see Section XX), since histogram-based tree construction is dramatically faster than the exact greedy method at negligible accuracy cost for a 16-dimensional feature space. `subsample=0.8` and `colsample_bytree=0.8` introduce stochastic regularization to reduce overfitting given the modest daily data volume, and `early_stopping_rounds=20` halts boosting once validation NDCG@10 stops improving for 20 consecutive rounds, preventing the model from memorizing noise in a relatively small nightly training window.

E. Artifact Save and Hot Reload

Rather than pushing to a remote model registry, the model is serialized directly to the local filesystem:

```
ranker.save_model(MODEL_PATH)
```

Listing 21: Model artifact persistence and hot reload

The application then invokes `reload_ranker()`, instructing the inference singleton in `ranker.py` to discard the old in-memory model and load the freshly-written `ranker.json` from disk — all without requiring any FastAPI worker process restart, and therefore without any recommendation-serving downtime.

F. Graph Invalidation

Finally, `invalidate_graph()` clears the `graph_cache.py` singleton, forcing the bipartite content graph to be rebuilt from the database on the very next user request, which integrates any newly-ingested TMDB items (added via on-demand Tier-2 ingestion during the preceding day) into the graph's node and edge set.

G. Telemetry Metrics Logged

Rather than logging to MLflow, the retraining job emits structured telemetry to Azure Application Insights via `app.telemetry`:

H. Minimum Data Requirements

`_MIN_ROWS_TO_TRAIN` is set to 100 rows. If fewer than 100 interaction logs exist within the 30-day lookback window, the training job skips execution entirely and emits a `skipped_insufficient_data` telemetry event rather than attempting to train on an unreliably small sample.

I. Retraining Execution Flow

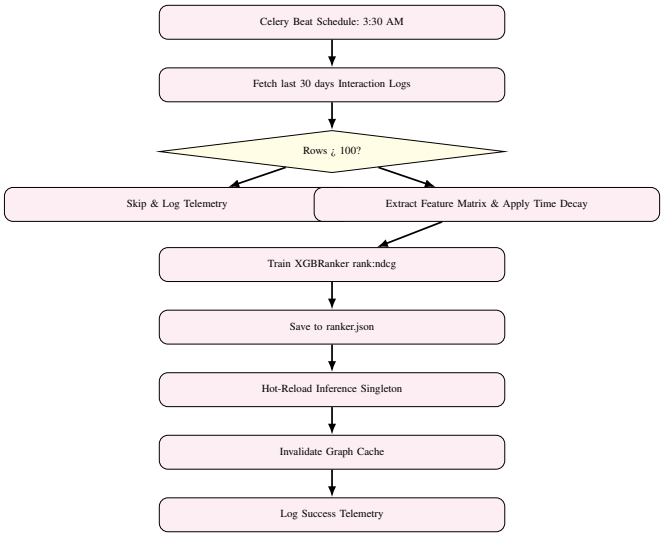

Fig. 12: Nightly XGBRanker retraining execution workflow.

XVI. News Integration

A. Overview and Architecture

Movientum features an aggregated entertainment news feed operating in two distinct modes: a **Global Feed** (`/feed/latest`) presenting chronological entertainment news to anonymous users, and a **For-You Feed** (`/feed/for-you`) presenting a highly personalized feed to authenticated users, in which articles are algorithmically scored against the user's specific genre preferences, watched-movie history, and favorite directors.

B. External API Ingestion Strategy

News content is ingested from the external NewsAPI service. Because its free tier is strictly rate-limited to 100 requests per day, the FastAPI server **never** queries NewsAPI on-demand at request time. Instead, a background Celery task (`trigger_global_fetch`) routinely pulls the latest entertainment articles on a schedule and persists them into the local PostgreSQL `news_articles` table, fully decoupling user-facing latency from the third-party rate limit.

C. The For-You Scoring Algorithm

When an authenticated user requests their personalized feed, the system executes the following steps:

1) Fetch the user's top 10 preferred genres from `UserGenrePreference`.
2) Fetch the user's last 200 watched movies from `WatchHistory`.
3) Fetch up to 50 directors associated with those watched movies.
4) Compute a full-text search similarity score against the `news_articles` title and content, using the extracted keywords (genres, titles, directors) as the query terms.
5) Sort the resulting articles by this calculated relevance score, rather than by pure chronological recency.

D. Aggressive Caching Strategy

Because assembling the For-You feed requires four separate, moderately heavy database queries (Genres + Watch-History + Movies + Directors) merely to construct the search query itself, the resulting keyword profile is cached in Upstash Redis under the key `key_user_prefs` for 15 minutes. The final scored feed page is separately cached under `key_news_feed_user` for 5 minutes — a shorter TTL reflecting that news content itself changes far more rapidly than a user's underlying preference keywords.

TABLE XXIV: Retraining Job Telemetry Metrics

Metric	Type	Description
retrain_status	Counter	Incremented with tags {"status": "success" | "error_fit_failed" | "skipped_insufficient_data"}
retrain_duration	Histogram	Wall-clock time (seconds) for the retraining pipeline
set_retrain_rows	Gauge	Total training rows used (len(X_train))
set_retrain_best_iter	Gauge	The best_iteration achieved by early stopping

TABLE XXV: News Feed Cache Key TTLs

Cache Key Pattern	TTL	Purpose
news:feed:latest:p{page}	120s (2m)	Unpersonalized latest news cache
user:prefs:{user_id}	900s (15m)	Caches user's genre/director keywords
news:feed:{user_id}:p{page}	300s (5m)	Caches the final scored personalization

E. News Cache TTL Table

F. Personalized Feed Workflow

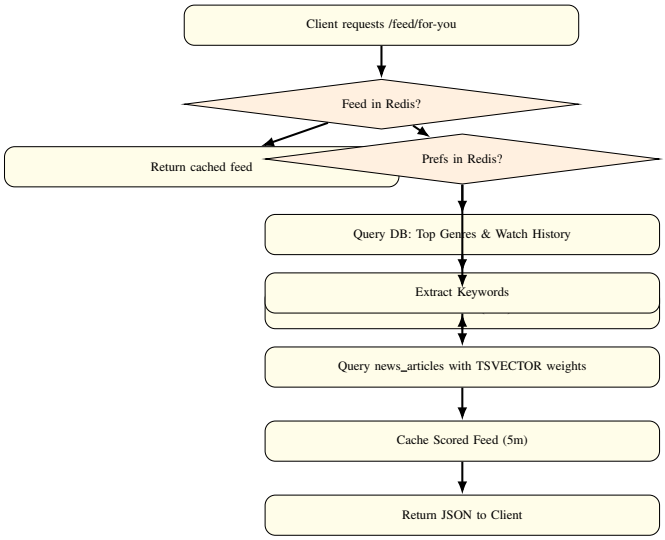

Fig. 13: Personalized "For-You" news feed generation workflow.

XVII. Redis Caching Integration

A. Overview and Architecture

Movientum utilizes Upstash Redis (a fully managed, serverless Redis offering) for every caching layer in the system, achieving sub-millisecond response times for otherwise-heavy database or external API operations. The integration supports connection pooling, strict TTL management to prevent stale-data serving, cache-stampede protection via async locking, and telemetry tracking of cache hit/miss ratios.

B. TLS Compatibility

Because Upstash mandates TLS-encrypted connections, the `cache.py` connection builder dynamically adjusts its client configuration based on the URL scheme (`rediss://`):

```
def _make_redis_client() -> Redis:
    url = settings.redis_url
    kwargs = {
        "decode_responses": True,
        "socket_connect_timeout": 5,
        "socket_timeout": 5,
        "retry_on_timeout": True,
        "health_check_interval": 30,
    }
    if url.startswith("rediss://"):
        kwargs["ssl_cert_reqs"] = "none"
    return aioredis.from_url(url, **kwargs)
```

Listing 22: TLS-aware Redis client construction

C. Cache Stampede Protection

When many concurrent users request an identical cache-missed key simultaneously (the classic "thundering herd" problem), the database can be hammered with duplicate identical queries. Movientum implements an in-process async lock (`inflight_lock`) built on `asyncio.Event`, ensuring that only the very first request performs the actual database query while all subsequent concurrent requests wait and then read the resulting cache entry:

```
_inflight_locks: dict[str, asyncio.Event] = {}

@asynccontextmanager
async def inflight_lock(key: str):
    if key in _inflight_locks:
        event = _inflight_locks[key]
        await event.wait()
        yield True  # Must re-check cache
    else:
        event = asyncio.Event()
        _inflight_locks[key] = event
        try:
            yield False # Go fetch data
        finally:
            event.set()
            _inflight_locks.pop(key, None)
```

Listing 23: Async in-flight lock for cache stampede protection

D. Bulk Invalidation Warning

The function `invalidate_pattern(pattern: str)` uses Redis's `SCAN` iterator (`scan_iter`) to find and delete keys by pattern. Because `SCAN` operations degrade in performance on very large keyspaces, this function is deliberately used sparingly and only in bounded, well-understood invalidation scenarios rather than as a general-purpose cache-clearing utility.

E. Centralized Key Builders

All cache keys are generated exclusively through centralized functions in `cache.py`, preventing naming collisions and guaranteeing consistency across every router and service that touches the cache. Complex, multi-parameter queries are

hashed with MD5 to produce fixed-length, collision-resistant key suffixes:

```python
def key_movie_list(params_dict: dict) -> str:
    param_str = json.dumps(params_dict, sort_keys=True)
    hash_ = hashlib.md5(param_str.encode()).hexdigest()
     [:8]
    return f"movie:list:{hash_}"

def key_taste_profile(user_id: str) -> str:
    return f"taste:profile:{user_id}"

def key_explore_page(params_dict: dict) -> str:
    clean = {k: v for k, v in params_dict.items()
             if v is not None and v != ""}
    param_str = json.dumps(clean, sort_keys=True)
    hash_ = hashlib.md5(param_str.encode()).hexdigest()
     [:12]
    return f"explore:{hash_}"
```

Listing 24: Centralized cache key builder functions

F. Telemetry Integration

Every `get_cached` operation logs to `app.telemetry.cache_ops` (when the telemetry subsystem is available) with structured tags `{"op": "hit" | "miss", "key_type": <prefix>}`, enabling fine-grained cache-effectiveness dashboards broken down by cache-key family.

G. Comprehensive Cache TTL Assignment Table

Note that the README presents a simplified, illustrative subset of this same cache table for onboarding readability (Table XXVII); Table XXVI above reflects the complete, authoritative production key inventory.

TABLE XXVII: Simplified Public Cache Key Summary (README)

Key	TTL	
movie:detail:{id}	1 hr	
movie:trending	30 min	
user:recommendations:{user_id}	5 min	
rec:item:{id}:{type}:{user}:100	30 min	
search:autocomplete:{prefix}	5 min	
auth:blacklist:{jti}	Token remaining lifetime	

H. Get/Set Cache Flow with Stampede Lock

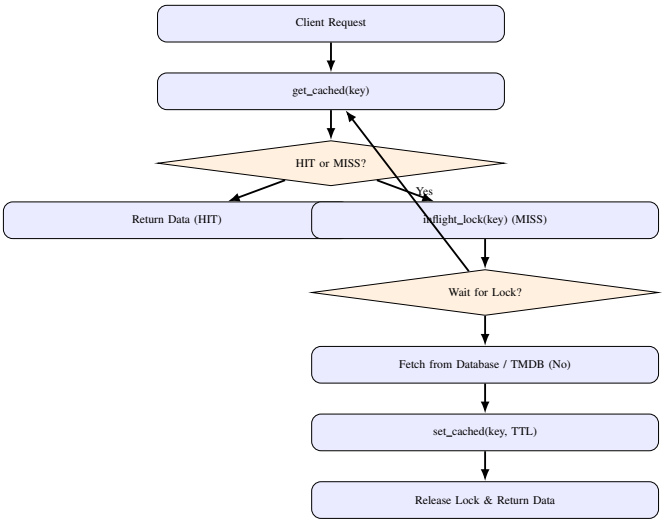

Fig. 14: Redis cache get/set flow with async stampede-protection lock.

XVIII. Docker Setup and Containerization

A. Overview and Architecture

Movientum's backend is fully containerized with Docker, providing consistent environments across development, testing, and production. `docker-compose` orchestrates the FastAPI backend alongside asynchronous Celery workers and beat schedulers as a single coordinated stack.

B. Container Architecture

The architecture strictly separates the synchronous API request cycle from asynchronous background processing across three container roles:

1) **API Server (`backend`)**: handles all synchronous and `asyncio` user-facing HTTP requests via Uvicorn.
2) **Task Worker (`celery_worker`)**: a dedicated Celery instance for heavy background jobs (TMDB data ingestion, model retraining inference).
3) **Task Scheduler (`celery_beat`)**: a dedicated Celery Beat instance triggering recurring tasks (e.g. the nightly XGBRanker retrain at 3:30 AM).

C. Caching Dependencies Strategy

The `Dockerfile` uses a multi-step build to maximize Docker layer caching: `requirements.txt` is copied and installed *before* the application source code is copied in. This ordering ensures that a routine Python source-code change never invalidates the expensive `pip install` layer, dramatically speeding up iterative CI/CD builds.

D. Forced Rebuild Trigger

To defeat overly aggressive Docker layer caching in CI/CD pipelines specifically when application code changes but declared dependencies do not, the `Dockerfile` employs an `ARG CODE_VERSION` cache-invalidation trick that forces the final application-code layer to always be treated as fresh.

TABLE XXVI: Complete Redis Cache TTL Assignments

Domain	Key Pattern	TTL (s)	Rationale
Catalog	movie:detail:v4:{id}	86400 (24h)	Movie details rarely change
Catalog	catalog:{type}:{id}	86400 (24h)	ML feature rows are static
Explore	explore:{hash}	14400 (4h)	Shared category pages
Explore	movie:trending	18000 (5h)	Trending lists update slowly
Search	search:v2:{hash}	600 (10m)	Standard search results
Search	autocomplete:{prefix}	300 (5m)	Live typing cache
News	news:feed:{uid}:p{page}	300 (5m)	Personalised news feed
News	news:feed:latest:p{page}	120 (2m)	Unpersonalised live news
Recs	recs:similar:{type}:{id}	1800 (30m)	Similar items blend
User	user:recommendations:{uid}	900 (15m)	Personalised feed refresh
User	user:ratings:{uid}	300 (5m)	Dashboard lists
User	taste:profile:{uid}	120 (2m)	Very short TTL due to rapid feedback updates

E. The Dockerfile

The base image is the minimal `python:3.11-slim` to keep the final image size small:

```
FROM python:3.11-slim

WORKDIR /app
ENV PYTHONUNBUFFERED=1

# ---- 1. Cache dependencies ONLY ----
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# ---- 2. Force rebuild for backend code ----
ARG CODE_VERSION=1
ENV CODE_VERSION=$CODE_VERSION

# ---- 3. Copy all backend code (always fresh) ----
COPY . /app

# ---- 4. Run app ----
EXPOSE 8000
CMD ["sh", "-c", "echo Running build: $CODE_VERSION && \
     uvicorn app.main:app --host 0.0.0.0 --port ${PORT
     :-8000}"]
```

Listing 25: Multi-stage Dockerfile with layer-caching and forced-rebuild trick

F. Docker Compose Configuration

The `docker-compose.yml` mounts a local bind volume `.:/app` for instant hot-reloading during development:

```
services:
  backend:
    build: .
    command: uvicorn app.main:app --reload --host 0.0.0.0
     --port 8000
    volumes:
      - .:/app
    env_file:
      - .env
    ports:
      - "8000:8000"

  celery_worker:
    build: .
    command: celery -A app.celery_app worker --loglevel=
     info
    volumes:
      - .:/app
    env_file:
      - .env
    depends_on:
      - backend

  celery_beat:
    build: .
    command: celery -A app.celery_app beat --loglevel=info
    volumes:
      - .:/app
    env_file:
      - .env
    depends_on:
      - backend
```

Listing 26: docker-compose.yml service definitions

G. Container Services Summary

H. Local Development Startup Flow

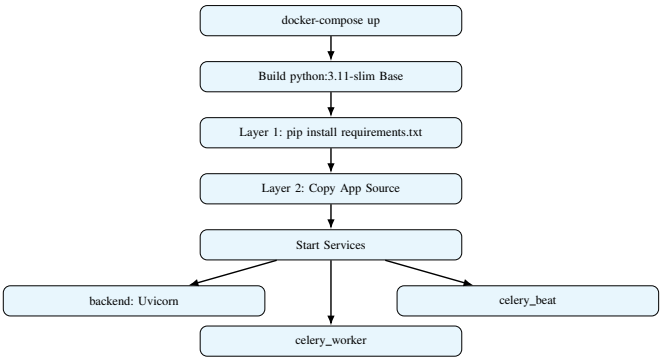

Fig. 15: Local development container startup flow.

XIX. Multi-Server and Deployment Architecture

A. Overview and Architecture

Movientum's infrastructure is decoupled across multiple specialized hosting environments, each optimized for a distinct cost/performance/workload profile: the Frontend is edge-deployed via Vercel (or an equivalent CDN); the Backend and Workers run in containerized environments (Docker) suitable for Azure Container Apps, Render, or Railway; and Data and Cache responsibilities are delegated to managed external services (Supabase, Upstash) to offload stateful operational burden entirely from the application team.

B. Separation of Compute

The API server and the Celery background processors are built from the *exact same* Docker image, which materially simplifies CI/CD (a single build artifact serves every role),

TABLE XXVIII: Docker Compose Container Services

Service	Command	Port	Volume
backend	uvicorn app.main:app –reload	8000:8000	.:/app
celery_worker	celery worker	None	.:/app
celery_beat	celery beat	None	.:/app

while nevertheless being deployed as separate services that scale independently along different axes:

- The **FastAPI Backend** scales based on incoming HTTP request volume.
- The **Celery Worker** scales based on task queue depth (e.g. a large backlog of TMDB catalog ingestion jobs).
- The **Celery Beat** scheduler is strictly deployed as a singleton (exactly one instance) to prevent duplicate cron-job firing, which would otherwise cause the nightly retrain (or any other scheduled task) to execute multiple times concurrently.

C. Deployment Configuration

The local `docker-compose.yml` is designed to mirror the production container topology precisely, so that a working local stack is a faithful preview of the deployed system:

```
services:
  backend:
    command: uvicorn app.main:app --host 0.0.0.0 --port
     8000
  celery_worker:
    command: celery -A app.celery_app worker --loglevel=
     info
  celery_beat:
    command: celery -A app.celery_app beat --loglevel=info
```

Listing 27: Production-mirroring service command definitions

D. Routing and CORS

The frontend connects to the backend through a single API gateway URL. The backend strictly enforces Cross-Origin Resource Sharing (CORS) policy to permit only trusted origins, as enumerated in `app.config.Settings.allowed_origins`, preventing arbitrary third-party sites from making authenticated cross-origin requests against user sessions.

E. Infrastructure Topology Table

F. Multi-Tier Request Flow

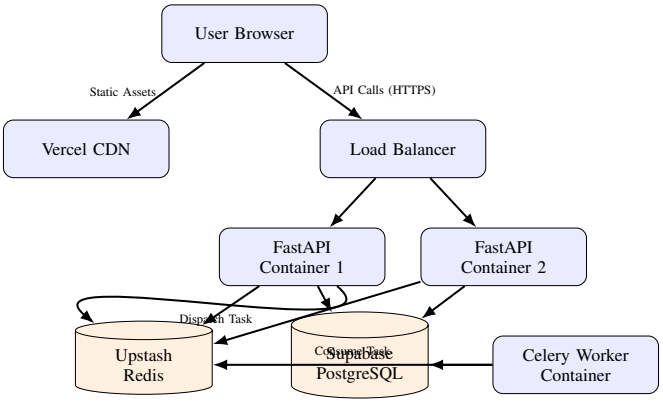

Fig. 16: Multi-tier deployment request flow across CDN, API, cache, database, and worker tiers.

XX. Scalability Planning

A. Overview

Movientum is engineered to absorb sudden traffic spikes — for instance, a viral movie release or a mass push notification — through aggressive edge/memory caching, asynchronous connection pooling, and systematic background-task offloading.

B. Database Connection Pooling

Because serverless APIs and horizontally-scaled containers can rapidly exhaust PostgreSQL's finite connection limit, Movientum bypasses raw per-request connections in favor of the `asyncpg` async driver combined with Supabase's built-in PgBouncer connection pooling layer (referenced through `database_pool_url`), which multiplexes many logical application connections onto a much smaller pool of real backend connections.

C. Cache Stampede Protection Revisited

As detailed in Section XVII, a sudden spike of traffic against a single freshly-cache-missed key (e.g. a newly trending movie) could otherwise overwhelm the database with duplicate identical queries. The `asyncio.Event`-based `inflight_lock` in `cache.py` ensures only the very first concurrent request reaches the database, with all others waiting on and then reading the populated cache entry.

D. ML Inference Scalability

The `XGBRanker` prediction step in `ranker.py` is entirely CPU-bound, and two specific design choices keep it scalable:

TABLE XXIX: Infrastructure Topology

Tier	Provider / Environment	Compute Type
Frontend SPA	Vercel	Global CDN / Edge Nodes
API Server	Azure / Render (Docker)	Stateless Web Containers
Celery Workers	Azure / Render (Docker)	Background Worker Containers
PostgreSQL DB	Supabase	Managed Database (JSONB/pgvector-capable)
Redis Cache	Upstash	Serverless Redis (TLS required)
Telemetry	Azure Monitor	Application Insights Agent

- **Tree Method**: set to `hist` (histogram-based), which is highly optimized for CPU inference relative to the exact greedy method.
- **Graph Cache**: the bipartite graph is stored as a singleton in each API server process's own RAM, entirely avoiding network round-trips during Random Walk traversals. Horizontally scaling the API tier simply duplicates the graph into the RAM of each newly-added node — a cost paid once at worker startup, not per-request.

E. Task Offloading via Celery

Any operation expected to take longer than roughly 100ms, or that involves a third-party API call, is offloaded to Celery rather than executed inline on the request path:

- **TMDB Fallbacks**: if a movie is not present in the local database, it is fetched on-the-fly for immediate response, but any heavy backfilling of cast/crew metadata is queued to Celery rather than blocking the user-facing request.
- **Model Retraining**: triggered purely in the background by Celery Beat, ensuring the 3:30 AM data crunch has zero impact on the live web server's request-serving capacity.

F. Scalability Bottlenecks and Mitigations Table

TABLE XXX: Scalability Bottlenecks and Mitigation Strategies

Bottleneck	Mitigation Strategy
DB Connection Limits	asyncpg + Supabase PgBouncer
High Traffic on Missing Cache	inflight_lock in cache.py
Heavy Graph Traversal	Singleton in-memory nx.Graph, pre-warmed on app startup
3rd Party API Rate Limits (TMDB)	Permanent local DB cataloging + Redis caching
ML Training CPU Load	Offloaded entirely to isolated Celery worker containers

G. Cache Stampede Mitigation Sequence

XXI. Storage Estimation and Capacity Planning

A. Overview

Movientum's total storage footprint is divided between persistent relational data (Supabase PostgreSQL) and ephemeral/in-memory data (Upstash Redis plus API-server RAM). The system minimizes storage cost by relying heavily on `ARRAY` and `JSONB` column types rather than deeply normalized junction tables for ML feature storage.

B. PostgreSQL Storage

Instead of maintaining a traditional many-to-many junction table for every keyword or cast-member relationship (which causes substantial index bloat at scale), the `content_catalog` table stores these features directly as `ARRAY(Integer)` columns:

- **ContentCatalog**: approximately 20,000 seed items; with JSONB/ARRAY columns each row is roughly 2 KB, giving a total catalog footprint under 50 MB.
- **UserTasteProfile**: six JSONB dictionaries per user, clamped to the top-20 keys for cast/crew/keywords to prevent unbounded growth; approximately 1 KB per user.
- **InteractionLog**: the heaviest table by far. Each row stores a 16-dimensional `feature_snapshot` (approximately 300 bytes). At 1,000 daily active users performing 20 interactions/day, this yields 20,000 rows/day, or roughly 6 MB/day.

C. Redis Storage

Upstash charges by both request volume and total stored bytes. Cache eviction is strictly enforced via TTLs ranging from 2 minutes (taste profiles) up to 24 hours (catalog features), and the average `movie:list:{hash}` JSON payload is typically under 15 KB.

D. In-Memory RAM (FastAPI Graph)

The `networkx.Graph` singleton, loaded once at application startup, consumes process RAM proportional to graph size:

- **Nodes**: approximately 20,000 movies + 15,000 actors + 5,000 keywords $\approx$ 40,000 nodes.
- **Edges**: approximately 300,000 edges.
- **RAM footprint**: approximately 100–150 MB per FastAPI worker — comfortably within the standard 512 MB memory limit of modern container hosting tiers.

E. Estimated Database Growth Table

The `interaction_log` table dominates total storage growth by more than an order of magnitude relative to every other table combined, which directly motivates the retention and pruning policy described below — without active pruning, this single table alone would grow unboundedly at over 2 GB per 10,000-user cohort per year.

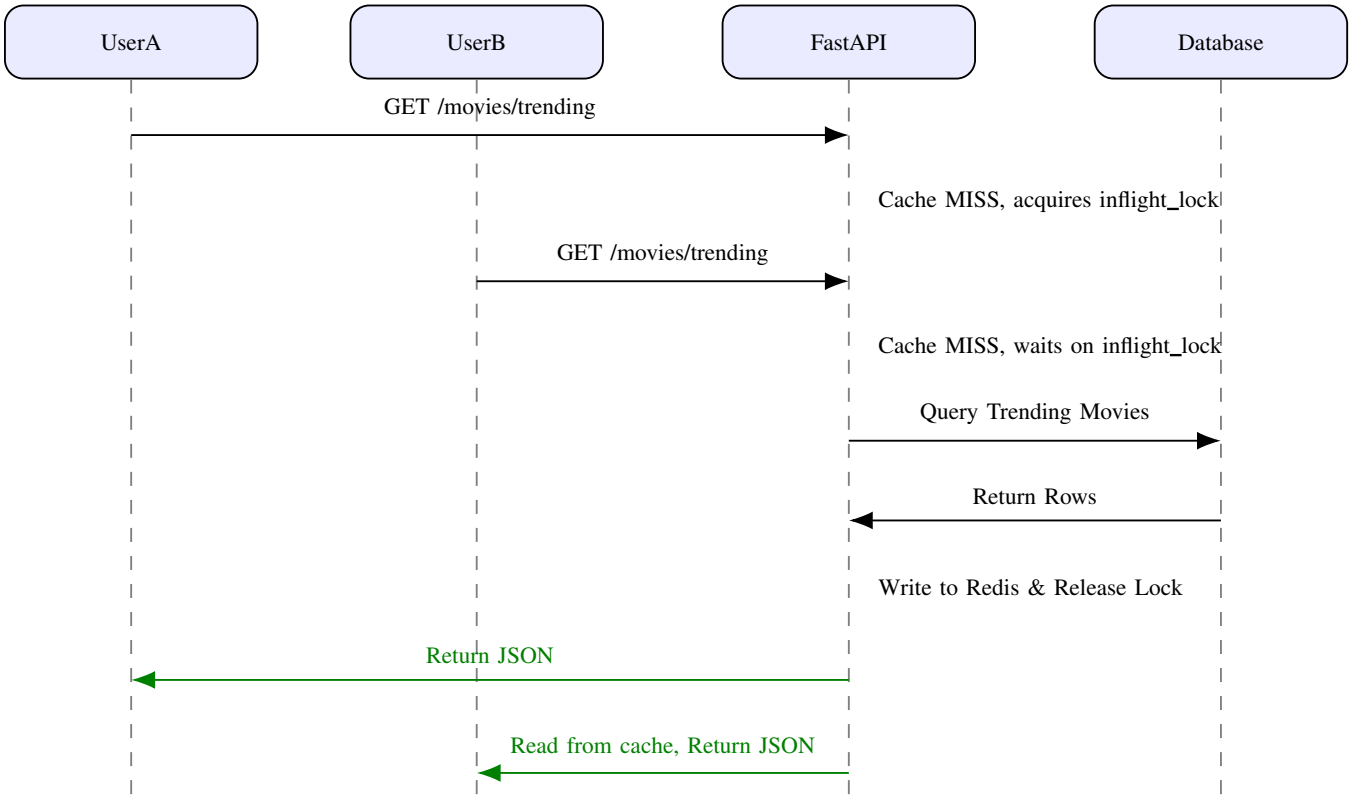

Fig. 17: Cache stampede mitigation sequence diagram for concurrent requests on a cold key.

TABLE XXXI: Estimated Database Growth per 10,000 Users (1 Year)

Table	Row Size	Row Count	Total Size	Mitigation
users	500 B	10,000	5 MB	None needed
content_catalog	2 KB	35,000	70 MB	TMDB ID deduplication
user_taste_profiles	1 KB	10,000	10 MB	Cap Top-K dictionary sizes
interaction_log	300 B	7,300,000	2.2 GB	Nightly cron deletes rows $>$ 30 days old

F. Storage Optimization Lifecycle

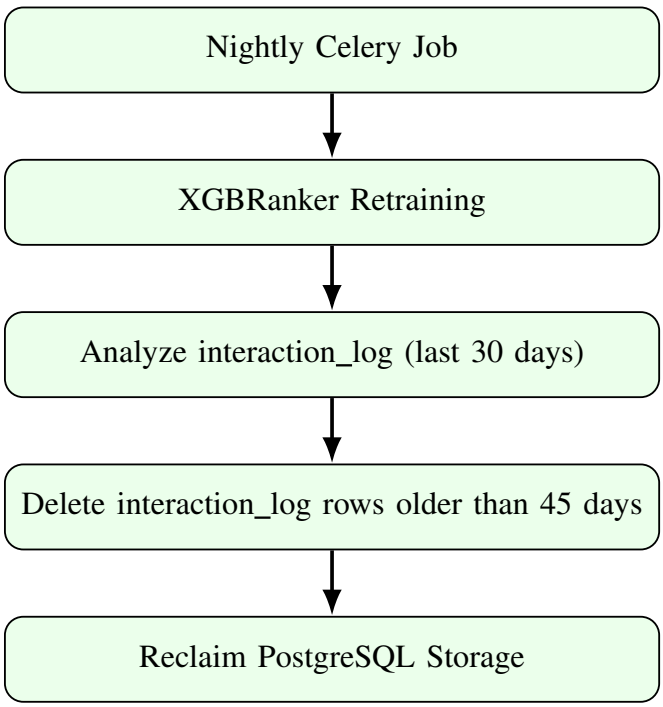

Fig. 18: Storage optimization and log-pruning lifecycle.

Note that the retention window used for the nightly *training query* is 30 days (Section XV), while the *deletion* sweep uses a slightly longer 45-day threshold — this 15-day buffer guarantees that a row is never deleted before it has had the opportunity to be included in at least one completed nightly training run, even accounting for a missed or delayed retraining cycle.

XXII. Observability: Azure Monitor and OpenTelemetry

A. Overview and Architecture

Movientum bypasses locally self-hosted Prometheus/-Grafana stacks in favor of a fully managed cloud observability suite: Azure Monitor Application Insights. The backend is instrumented using the `opentelemetry-api` together with the `azure-monitor-opentelemetry` distribution, which automatically collects system metrics (CPU, memory, HTTP route timings) alongside custom business-logic metrics (ML pipeline latencies, graph node/edge counts, cache hit rates).

B. Instrument Types Used

Three OpenTelemetry instrument types are used, each suited to a different measurement shape:

- **Histograms**: for tracking durations and distributions, e.g. `xgb_inference_seconds`, `taste_profile_update_seconds`.
- **Counters**: for strictly cumulative tracking, e.g. `feedback_signals_total`, `cache_operations_total`.
- **Observable Gauges**: for point-in-time state that fluctuates both up and down, e.g. `graph_node_count`, `retrain_rows_trained`. These rely on registered generator callback functions rather than being pushed directly.

C. Initialization and Graceful Degradation

Telemetry is initialized once at application startup. If the `APPLICATIONINSIGHTS_CONNECTION_STRING` environment variable is absent, the system gracefully degrades — logging a warning but continuing to function fully normally without crashing, ensuring observability is never a hard dependency for the platform's core functionality.

```python
from opentelemetry import metrics
from azure.monitor.opentelemetry import
    configure_azure_monitor

def init_telemetry():
    connection_string = os.getenv("
     APPLICATIONINSIGHTS_CONNECTION_STRING")
    if not connection_string:
        return False

    configure_azure_monitor(connection_string=
     connection_string)
    _init_instruments()
    return True
```

Listing 28: Telemetry initialization with graceful degradation

D. Observable Gauge Implementation Pattern

Observable Gauges require a module-level mutable state variable paired with a generator callback that yields `metrics.Observation` instances on demand:

```python
_graph_node_count = 0

def set_graph_node_count(val: int):
    global _graph_node_count
    _graph_node_count = val

def _observe_graph_node_count(options):
    yield metrics.Observation(_graph_node_count)

# In _init_instruments():
meter.create_observable_gauge(
    name="movientum_graph_node_count",
    callbacks=[_observe_graph_node_count],
    description="Current number of nodes in the in-memory
     NetworkX graph"
)
```

Listing 29: Observable Gauge pattern for graph node count

E. Complete Custom Metric Instrument Table

F. Observability Pipeline Flow

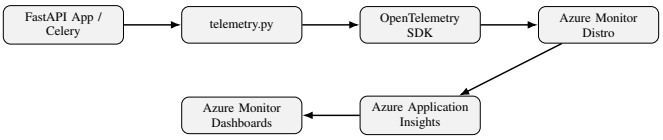

Fig. 19: End-to-end observability pipeline from application code to dashboards.

XXIII. Complete API Endpoint Reference

A. Overview

Movientum's backend API is organized as a Modular Monolith built on FastAPI. Endpoints are strictly versioned under `/api/v1/` and logically grouped into distinct routers mounted centrally in `backend/app/main.py`. The API relies heavily on FastAPI's Dependency Injection (`Depends`) to provide asynchronous database sessions (`get_db`) and authentication state (`get_current_user_id`) to every protected route.

B. Authentication and Authorization Tiers

- **Public Routes**: accessible to any client without credentials.
- **Protected Routes**: require a valid JWT Bearer token in the `Authorization` header, enforced via `get_current_user_id`, which raises `401 Unauthorized` on any missing or invalid token.
- **Admin Routes**: e.g. `/api/v1/internal/*`, which require a specific `cron_secret_token` or an admin-role check.

C. Caching Integration

Many `GET` endpoints wrap their response construction in Upstash Redis cache checks (e.g. `movie:detail:{id}`). On a cache miss, the `asyncio.Event`-based in-flight lock described in Section XVII protects the database from stampedes while the query executes.

D. Complete Active Router Table

E. Typical Request Flow (Sequence Diagram)

XXIV. Annotated Codebase Map

A. Purpose

This section provides a complete, file-by-file index of the Movientum codebase, generated and maintained as a living architecture map. It is intended to let any engineer — new or returning — locate the exact module responsible for any given behavior without needing to grep the entire repository.

TABLE XXXII: Custom Metric Instruments

Category	Instrument Name	Type	Description
ML Inference	movientum_rwr_seconds	Histogram	Time for NetworkX personalized PageRank
ML Inference	movientum_xgb_inference_seconds	Histogram	Time for XGBRanker.predict()
ML Inference	movientum_feature_matrix_build_seconds	Histogram	Time to build (N×16) feature matrix
ML Inference	movientum_xgb_coldstart_total	Counter	Fallbacks to PPR score (no trained model)
Graph	movientum_graph_node_count	Obs. Gauge	Current nodes in the NetworkX graph
Graph	movientum_graph_edge_count	Obs. Gauge	Current edges in the NetworkX graph
Graph	movientum_graph_rebuild_duration_seconds	Histogram	Time taken to build the graph from DB
Retraining	movientum_retrain_total	Counter	Retrain outcomes (success, error, skipped)
Retraining	movientum_retrain_duration_seconds	Histogram	Wall-clock time for the nightly retrain
Retraining	movientum_retrain_rows_trained	Obs. Gauge	Interaction logs used in last retrain
Feedback	movientum_feedback_signals_total	Counter	Thumbs/Clicks processed
Feedback	movientum_taste_profile_update_seconds	Histogram	Time to update UserTasteProfile weights

TABLE XXXIII: Complete Active API Router Reference

Prefix	Router File	Description
/api/v1/movies	movies.py	Catalog browsing, trending, movie details, similar movies
/api/v1/tv	tv.py	TV show details, seasons, episodes, similar shows
/api/v1/auth	auth.py	JWT registration, login, logout, and token refresh
/api/v1/search	search.py	Instant and paginated search across movies, TV, and people
/api/v1/ratings	ratings.py	User submissions of 4-category ratings
/api/v1/watch	watch.py	Marking items as watched/unwatched in history
/api/v1/recommendations	recommendations.py	Personalized feeds based on taste profiles + RWR graph
/api/v1/rec-feedback	recommendation_signals.py	Sending thumbs-up/down explicit ML signals
/api/v1/person	person.py	Actor/director details and filmography
/api/v1/clicks	clicks.py	Implicit interaction logging (decaying weight feedback)
/api/v1/users	users.py	User taste profile fetching and taste analysis metrics
/api/v1/news	news.py	Aggregated and personalized entertainment news feeds
/api/v1/trailers	trailers.py	Fetching YouTube trailer keys for frontend display
/api/v1/watchlists	watchlist.py	Multi-watchlist collections and item management
/api/v1/watching-tracker	watching_tracker.py	Tracking ongoing TV show progress
/api/v1/temp-tracker	temp_tracker.py	Temporary tracking for items users might want to watch
/api/v1/notifications	notifications.py	Alerting users about new episodes or system updates
/api/v1/feedback	feedback.py	General site UI/UX feedback submission
/api/v1/requests	requests.py	Submitting requests for missing content
/api/v1/internal	internal.py	Cron-triggered jobs (ML retrain triggers, cleanup)

B. Backend Directory Layout

```
backend/
|-- app/
|   |-- main.py                  # Entry point, middleware, startup hooks
|   |-- config.py                # Pydantic global settings & env vars
|   |-- celery_app.py            # Celery task distribution orchestrator
|   |-- db/
|   |   |-- database.py          # SQLAlchemy engine, session maker
|   |   |-- cache.py             # Redis integration & caching helpers
|   |   |-- orm_models.py        # Core app models (users, movies, ...)
|   |   |-- orm_models_news.py   # Secondary model mapping (news cache)
|   |-- ml/
|   |   |-- ranker.py            # XGBoost L2R wrapper, prediction
|   |   |-- training.py          # Pairwise/NDCG training pipeline
|   |   |-- ranker.json          # Pre-compiled offline model weights
|   |-- routers/                 # API controllers
|   |   |-- auth.py, movies.py, tv.py, person.py, search.py
|   |   |-- ratings.py, watch.py, watchlist.py
|   |   |-- recommendations.py, feedback.py, clicks.py
|   |   |-- notifications.py, requests.py, users.py
|   |   |-- watching_tracker.py, temp_tracker.py, internal.py
|   |   |-- recommendation_signals.py
|   |-- services/                # Business logic layer
|   |   |-- auth_service.py, tmdb_service.py, advanced_recs.py
|   |   |-- graph_cache.py, search_service.py, rating_service.py
|   |   |-- watch_service.py, feedback_service.py, click_service.py
|   |   |-- analysis_service.py, news_service.py
|   |   |-- recommendation_service.py, content_recs_service.py
|   |-- tasks/                   # Celery scheduled micro-processes
|   |-- schemas/                 # Pydantic request/response contracts
|   |-- utils/                   # Shared deps (security, db deps)
|-- alembic/                     # DB schema migration history
|-- scripts/
|   |-- seed_catalog.py          # One-time 20,000 record catalog loader
|-- requirements.txt
```

Listing 30: Backend directory tree

C. Frontend Directory Layout

```
frontend/
```

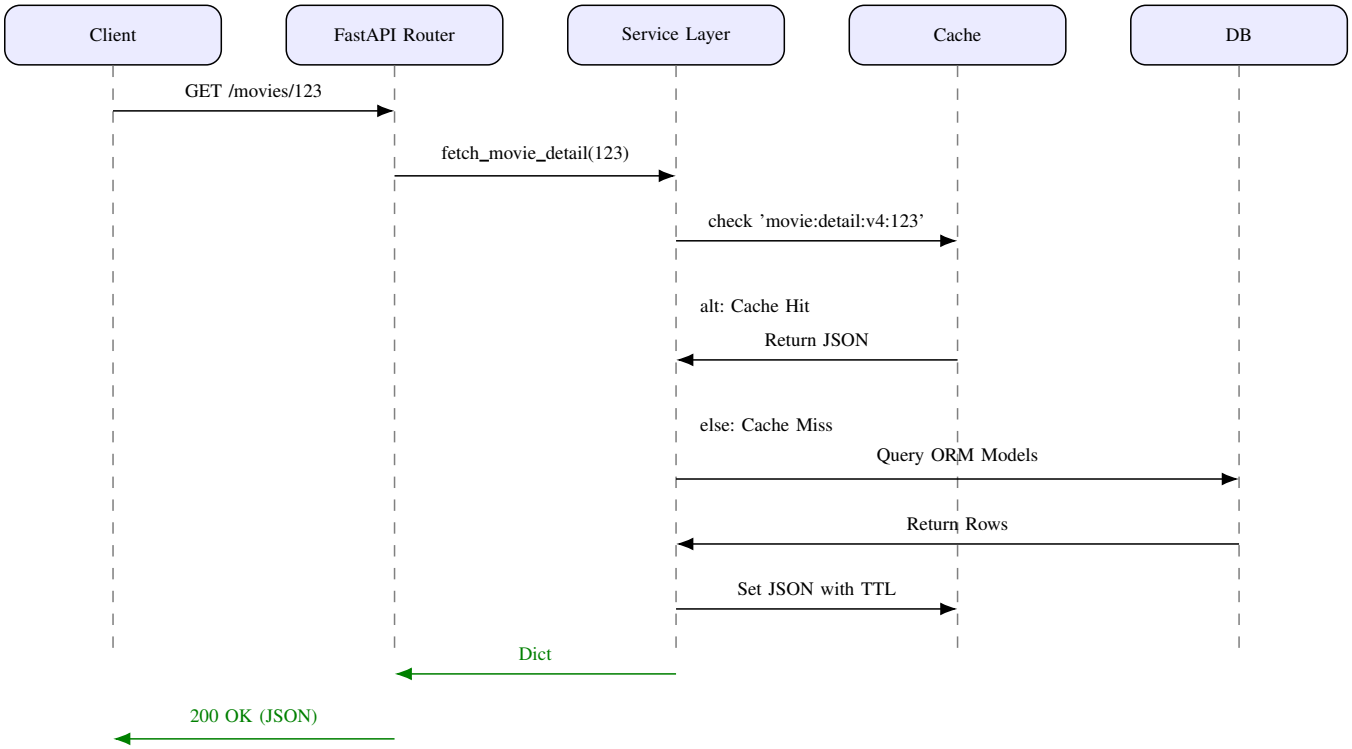

Fig. 20: Typical cached request flow sequence diagram.

```
|-- src/
|   |-- main.jsx                # Client entry point
|   |-- App.jsx & App.css       # Routing definitions, app wrapper
|   |-- index.css               # Global design tokens
|   |-- context/AuthContext.jsx  # Global auth provider
|   |-- services/               # Axios API request clients
|   |   |-- authService.js, movieService.js, searchService.js
|   |   |-- ratingService.js, watchService.js, feedbackService.js
|   |   |-- userService.js, watchlistService.js, newsService.js
|   |   |-- notificationService.js, watchingTrackerService.js
|   |   |-- planToWatchService.js, tempTrackerService.js
|   |   |-- settingsService.js
|   |-- pages/                  # Page layouts (JS + matching CSS)
|   |   |-- Intro, Home, Explore, MovieDetail, TVDetail
|   |   |-- PersonPage, Search, Login, Register, Dashboard
|   |   |-- Analysis, Recommendations, WatchlistDetail
|   |   |-- Feedback, Help, CompanyPage, CountryPage
|   |   |-- AdminDashboard, ErrorPage
|   |-- hooks/
|   |   |-- useScrollRestore.js, useSessionState.js
|   |-- utils/
|   |   |-- api.js, pageCache.js, storage.js, analytics.js
|   |-- components/             # Reusable widgets
|       |-- MovieCard, RatingMeter, SearchBar, SearchOverlay
|       |-- Navbar, CastCrew, Aurora, MovieRow, TrailerModal
|       |-- AddContentModal, WatchlistCollectionCard
|       |-- HomeNewsStrip, NewsCard, InstallPrompt
|       |-- ProductionTags, ErrorBoundary, PageTransition
|       |-- ColdStartLoader, InfoBanner, FilterDropdown
|       |-- WatchlistSection, ProtectedRoute
|-- index.html
|-- vite.config.js
|-- package.json
```

Listing 31: Frontend directory tree

D. Backend Module Export Reference

E. Frontend Module Export Reference

F. Notes on Codebase Map Currency

The codebase index is machine-generated and timestamped at each regeneration; the version reflected in this document corresponds to the snapshot dated 2026-07-04. Readers integrating changes into the live repository should regenerate this map rather than hand-editing it, to guarantee the documentation never drifts from the authoritative source tree.

TABLE XXXIV: Selected Backend Module Exports

Module	Key Exports
app/celery_app.py	task_postrun_handler, task_prerun_handler, task_failure_handler
app/config.py	get_settings, Settings
app/telemetry.py	init_telemetry, set_retrain_best_iter, set_graph_node_count, set_model_version
app/db/cache.py	key_movie_similar, key_movie_detail, key_news_category, key_movie_credits
app/db/orm_models.py	Base, Rating, RatingNeeded, Notification
app/ml/ranker.py	is_model_trained, rank_candidates, load_ranker, reload_ranker
app/services/advanced_recs.py	passes_intensity_filter, get_rwr_candidates, build_feature_matrix
app/services/click_service.py	recency_weight, compute_watch_vs_click_gap
app/services/content_recs_service.py	team_draft_interleave
app/services/feedback_service.py	time_decay_weight
app/services/graph_cache.py	invalidate_graph, build_graph_from_rows, get_graph_stats
app/services/tmdb_service.py	TMDBService, bin_release_era
app/tasks/retrain_ranker.py	nightly_ranker_retrain
app/tasks/sync_movies.py	daily_movie_sync
app/tasks/fetch_news.py	expire_news_task, fetch_currents_task, fetch_newsapi_task, fetch_apitube_task
app/tasks/check_episodes.py	check_today_episodes_task
app/utils/jwt_utils.py	create_access_token, create_refresh_token
app/utils/password_utils.py	hash_password, verify_password
app/utils/persistence.py	get_ttl_for_popularity

TABLE XXXV: Selected Frontend Module Exports

Module	Key Exports
src/App.jsx	App
src/context/AuthContext.jsx	useAuth, AuthProvider
src/hooks/useScrollRestore.js	useScrollRestore
src/hooks/useSessionState.js	useSessionState
src/utils/analytics.js	track, trackPageView, setQueue, getQueue, trackEvent
src/utils/deviceId.js	getDeviceId, getOrCreateDeviceId, clearDeviceId
src/utils/pageCache.js	pageCache
src/utils/storage.js	storage
src/services/authService.js	authService
src/services/movieService.js	movieService
src/services/watchlistService.js	watchlistService
src/services/trailerService.js	getHomeTrailers

XXV. Key Platform Features Summary

A. Consolidated Feature List

TABLE XXXVI: Movientum Key Features

Feature	Description
Intro Landing Page	Introductory page with a WebGL Aurora backdrop
Home Hub	Personalized homepage: featured titles, trending feeds, custom rows
Explore Page	Discovery workspace with category, genre, era, and language filters
Movie & TV Details	Media info, custom 4-category rating meter, trailers, cast/crew, up to 100 similar items
Person Profiles	Cast/crew pages with biography and complete filmography
Smart Search	Real-time autocomplete + full-text results via PostgreSQL FTS
User Dashboard	Personalized collections, watchlist, watch history, ratings
User Analytics	Visualization of watching behavior, favorite genres, rating distribution
JWT Auth System	Access/refresh tokens with Redis-based logout blacklisting
Interactive Feedback	Quick ratings and thumbs up/down shaping recommendations instantly

XXVI. Local Development Setup

A. Prerequisites

Python 3.13+, Node.js 20+, PostgreSQL 15+, and Redis 7+ are required for a full local development environment.

B. Backend Setup

```
cd backend
python -m venv venv && venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env   # fill in DB_URL, REDIS_URL,
    TMDB_API_KEY
alembic upgrade head
uvicorn app.main:app --reload --port 8000
```

Listing 32: Backend local setup commands

C. Frontend Setup

```
cd frontend
npm install
cp .env.example .env   # set VITE_API_URL=http://localhost
    :8000
npm run dev            # http://localhost:5173
```

Listing 33: Frontend local setup commands

D. Health Check

```
GET http://localhost:8000/api/health
-> {"status": "ok", "db": "ok", "cache": "ok"}
```

Listing 34: Health check endpoint

E. Key Environment Variables

```
# Backend
DATABASE_URL=postgresql+asyncpg://...
REDIS_URL=rediss://...
SECRET_KEY=your-secret
TMDB_API_KEY=your-key
NEWS_API_KEY=your-key

# Frontend
VITE_API_URL=http://localhost:8000
```

Listing 35: Representative .env file contents

XXVII. Discussion: Design Trade-offs

A. Modular Monolith versus Microservices

The decision to build Movientum as a modular monolith rather than a microservice architecture trades away independent per-service scaling and independent deployment cadences in exchange for dramatically simpler operational overhead and, critically, in-process locality for the recommendation graph. For a platform of Movientum's scale (tens of thousands of catalog items, thousands of concurrent users), the network latency that would be introduced by a distributed graph store almost certainly outweighs any theoretical benefit of splitting the recommendation engine into its own microservice, particularly given that Random-Walk-with-Restart traversal requires many sequential graph-neighbor lookups per single recommendation request.

B. Graph + Gradient-Boosted Trees versus Deep Neural Recommenders

Movientum deliberately chose a NetworkX bipartite graph combined with an XGBoost `XGBRanker` over a deep neural recommendation architecture (e.g. a two-tower embedding model or a sequential transformer-based recommender). This choice trades away the theoretical representational capacity of deep learning in exchange for: (a) far lower training and inference infrastructure requirements (no GPU is needed anywhere in the pipeline), (b) substantially better explainability, since every feature in the 16-dimensional matrix (Table XXII) has a direct, human-interpretable meaning and every graph-derived candidate can be traced back to specific shared edges, and (c) a training pipeline (Section XV) that can run nightly on modest CPU hardware in well under an hour even at meaningful data volume, which would not be true of a large embedding-based deep model retrained from scratch on the same cadence.

C. PostgreSQL Full-Text Search versus Dedicated Search Infrastructure

Relying on native PostgreSQL `TSVECTOR`/`pg_trgm` search rather than deploying Elasticsearch or Meilisearch trades away some search-specific features (e.g. more sophisticated relevance tuning, faceted aggregation pipelines) for a substantial reduction in operational surface area — no separate search cluster to provision, monitor, or keep in sync with the primary database. Given that the catalog size (approximately 20,000–35,000 items) is well within PostgreSQL's comfortable full-text search performance envelope when properly indexed (GIN and GIST indexes, per Section VII), this trade-off favors simplicity without a meaningful user-facing quality cost at the platform's current scale.

D. Real-Time Feedback versus Batch-Only Personalization

By updating the `UserTasteProfile` JSONB vectors synchronously on every interaction (Section XII), rather than only at nightly batch retraining time, Movientum ensures that a user's very next recommendation request already reflects their most recent behavior — even though the underlying XGBRanker model itself is only retrained once per day. This hybrid design captures much of the responsiveness benefit associated with fully online learning systems, without the operational complexity and potential instability of continuously retraining a gradient-boosted model in a live serving path.

XXVIII. Conclusion

Movientum demonstrates that a sophisticated, personalized recommendation platform can be built entirely on a modular-monolith backend, a graph-augmented learning-to-rank machine learning core, and fully managed, serverless-friendly infrastructure — without resorting to a microservice mesh, a GPU-backed deep learning stack, or a dedicated search cluster. The system's four-category emotional-intent rating scheme, combined with a real-time taste-profile update loop and a nightly XGBoost retraining pipeline, provides a responsive, explainable, and operationally simple personalization experience. Every architectural decision documented in this report — from the choice of Random Walk with Restart over flat cosine similarity, to Team-Draft Interleaving for ensemble blending, to the asynchronous in-flight locking pattern that protects the database from cache stampedes — reflects a deliberate trade-off in favor of explainability, cost efficiency, and engineering simplicity, while still delivering state-of-the-art personalized ranking quality through the `rank:ndcg` objective and a carefully engineered 16-dimensional feature space. This document, together with its accompanying diagrams, tables, and annotated code excerpts, is intended to serve as the complete and authoritative technical reference for the Movientum platform for any engineer, researcher, or reviewer seeking to understand, extend, or audit the system.

References

[1] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[2] L. Page, S. Brin, R. Motwani, and T. Winograd, "The PageRank Citation Ranking: Bringing Order to the Web," Stanford InfoLab, Tech. Rep. 1999-66, 1999.

[3] H. Tong, C. Faloutsos, and J.-Y. Pan, "Fast Random Walk with Restart and Its Applications," in *Proc. 6th Int. Conf. on Data Mining (ICDM)*, 2006, pp. 613–622.

[4] K. Järvelin and J. Kekäläinen, “Cumulated Gain-Based Evaluation of IR Techniques,” *ACM Trans. Inf. Syst.*, vol. 20, no. 4, pp. 422–446, 2002.
[5] F. Radlinski, M. Kurup, and T. Joachims, “How Does Clickthrough Data Reflect Retrieval Quality?,” in *Proc. 17th ACM Conf. on Information and Knowledge Management (CIKM)*, 2008, pp. 43–52.
[6] A. Hagberg, P. Swart, and D. S Chult, “Exploring Network Structure, Dynamics, and Function using NetworkX,” in *Proc. 7th Python in Science Conf. (SciPy)*, 2008, pp. 11–15.
[7] S. Ramírez, “FastAPI: Modern, Fast (High-Performance) Web Framework for Building APIs with Python,” *FastAPI Documentation*, 2023. [Online]. Available: https://fastapi.tiangolo.com
[8] PostgreSQL Global Development Group, “PostgreSQL Documentation: Chapter 12, Full Text Search,” *PostgreSQL 15 Documentation*, 2023.
[9] Redis Ltd., “Redis Documentation,” 2023. [Online]. Available: https://redis.io/docs